Docker Mastery: From Zero to DevOps Hero ��🐳 Part 1

Docker Learning Path

Contents

1 Docker Mastery: From Zero to DevOps Hero

Welcome to **Docker Mastery: From Zero to DevOps Hero** — a single, comprehensive, self-paced learning resource that takes you from a completely empty Windows laptop with no Docker installed to a confident, production-ready DevOps practitioner. This document is designed for absolute beginners, curious developers, sysadmins pivoting to DevOps, and experienced engineers who want a single source of truth for Docker. Read it linearly if you are new, or jump to any milestone using the Table of Contents if you already know the basics. Every milestone is self-contained, so you can pause and resume at any point without losing context.

> **Navigation Tip:** If you're viewing this in VS Code, Obsidian, Typora, or GitHub, a clickable Table of Contents will appear in the left sidebar automatically. Use it to jump between milestones. If you're reading as a PDF, use the Table of Contents below or the PDF bookmark

panel. For printing, page numbers will appear at the bottom-right of each page.

1.1 Table of Contents

        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 7: Container Data Management
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 8: Docker Networking
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
- Phase 3 — Composing (□→□)
    - Milestone 9: Docker Compose Fundamentals
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 10: Advanced Compose Patterns
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
- Phase 4 — Production Readiness (□ Advanced)
    - Milestone 11: Docker Registry & Image Distribution
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 12: Docker Security
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises

        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 13: Docker Logging, Monitoring & Debugging
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 14: Container Lifecycle & Process Management
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
- Phase 5 — Orchestration & DevOps (● Expert)
    - Milestone 15: Docker Swarm
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 16: Docker in CI/CD Pipelines
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 17: Advanced Docker Patterns & Best Practices
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project
        - Scenario
        - Checkpoint Quiz
    - Milestone 18: Docker for Microservices Architecture
        - Theory
        - Commands
        - Command Options/Flags
        - Examples
        - Command Exercises
        - Hands-On Assignment
        - Mini-Project

1.2 Phase 1 — Foundation (Beginner)

This phase builds your mental model of what containers are, gets Docker running on your Windows machine, and teaches you the fundamental CLI commands you will use every single day for the rest of your career. Do not rush this phase — the analogies and terminology you learn here will make every advanced topic later feel intuitive.

1.2.1 Milestone 1: Understanding Containers & Docker

Theory

Before you type a single Docker command, you need a rock-solid mental model of what a container actually is, how it differs from a virtual machine, and what problem Docker was invented to solve. This section is intentionally theory-heavy — the hands-on commands come in Milestone 2, but if you skip this section you will be memorizing commands without understanding them.

1.2.1.0.1 The Problem Docker Solves — “It Works on My Machine” Imagine you are a chef. You perfect a recipe in your own kitchen using your specific oven, your specific pans, your specific brand of flour, and your specific altitude. You email the recipe to a friend across the country. Your friend follows it exactly but the cake collapses. Why? Because their oven runs 20°F hotter, their flour has a different protein content, and they live at a higher altitude. The recipe is the same, but the environment is different.

Software has this exact problem. A developer writes an application on their Windows laptop with Node.js 20.5, connects it to PostgreSQL 15.2, and uses a specific version of a Linux utility installed system-wide. It runs perfectly. They ship the code to a colleague running Node.js 18, or to a Linux server missing that utility, and it explodes. This is the famous “it works on my machine” problem.

Docker solves this by letting you package your application *along with its entire environment* — the operating system libraries, the runtime, the dependencies, the configuration — into a single portable unit called a **container**. When you ship the container, you are shipping the kitchen, not just the recipe.

1.2.1.0.2 What Is Virtualization? **Virtualization** is a technique that lets one physical computer pretend to be many separate computers. It does this using a piece of software called a **hypervisor** (examples: VMware ESXi, Microsoft Hyper-V, Oracle VirtualBox, KVM). The hypervisor carves up the physical CPU, RAM, and disk and hands slices to each virtual computer, called a **Virtual Machine (VM)**.

Each VM contains a **complete operating system** — a full Linux or Windows install with its own kernel, its own drivers, its own init system, and every system service. On top of that OS, you install your application and its libraries. Booting a VM takes minutes because it is booting a full OS from a virtual BIOS upward, exactly like a physical computer.

Think of virtualization like a large apartment building where every apartment has its own furnace, its own water heater, its own electrical panel, and its own front door onto the street. Each apartment is fully independent, but that independence costs a lot of duplicated infrastructure.

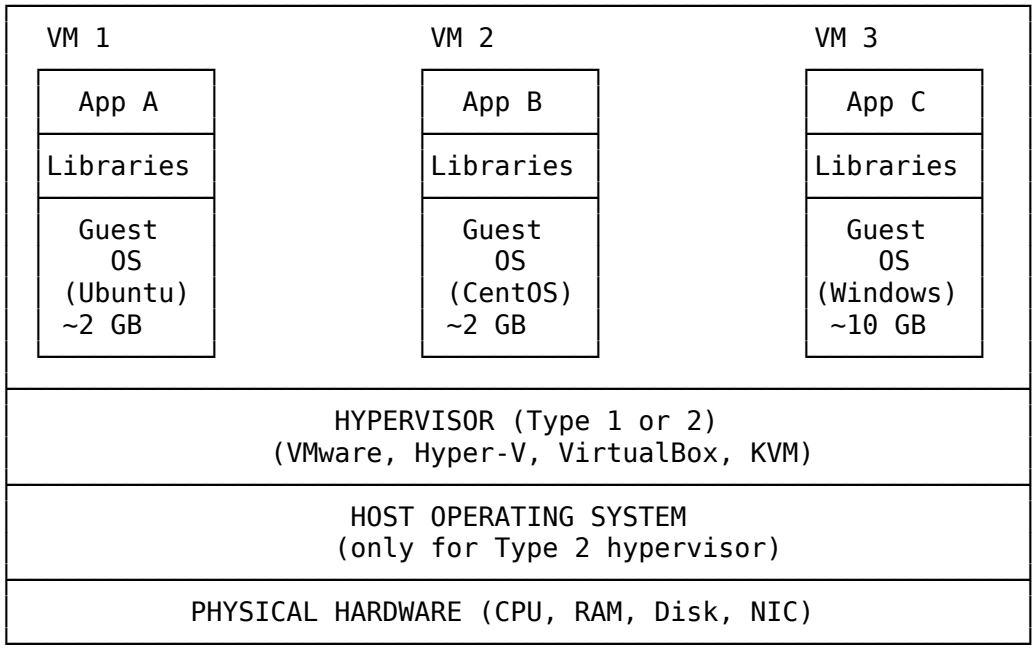
1.2.1.0.3 What Is Containerization? **Containerization** is a lighter-weight isolation technique. Instead of virtualizing an entire computer, containers share the **host operating system's kernel** and virtualize only the parts *above* the kernel — the filesystem, the network stack, the process list, and the user list. This is done using Linux kernel features called **namespaces** (which isolate what a process can *see*) and **cgroups** (which limit what a process can *use*, like CPU and memory).

Because containers share the host kernel, they start in milliseconds, use a fraction of the RAM of a VM, and have almost zero performance overhead. But because they share the kernel, a Linux container needs a Linux kernel — you cannot run a Windows container on a Linux kernel or vice versa. (On Windows, Docker Desktop uses a lightweight Linux VM called WSL2 to provide the Linux kernel for Linux containers.)

Think of containerization like a single-family house divided into apartments that share the main furnace, water heater, and electrical panel of the house. Each apartment has its own private rooms, its own front door, and its own kitchen — but the underlying infrastructure is shared, saving enormous amounts of duplicated equipment.

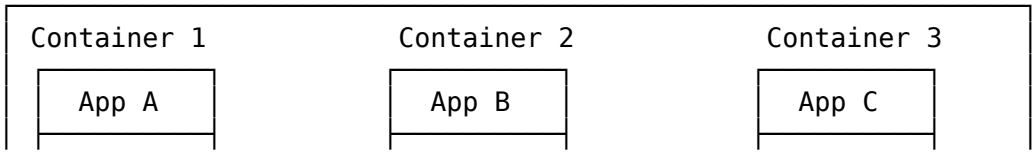
1.2.1.0.4 ASCII Diagram — Virtual Machine Architecture

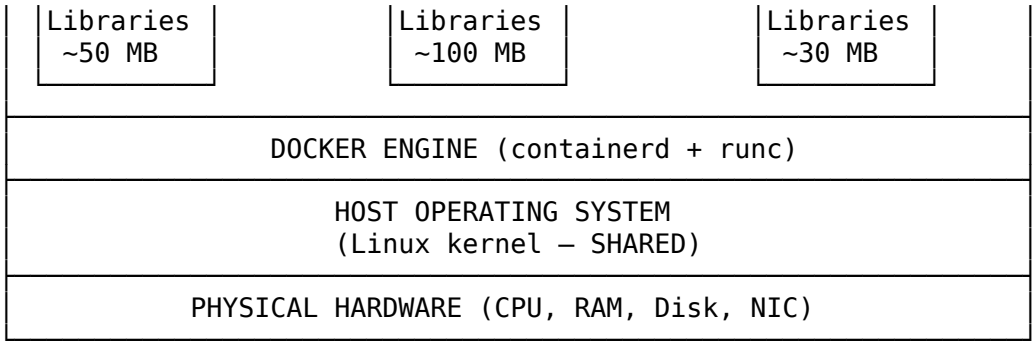
<!-- CODE -->



1.2.1.0.5 ASCII Diagram — Container Architecture

<!-- CODE -->





Notice how the container stack is dramatically thinner. No guest OS per container, no hypervisor. That is where all the speed and efficiency come from.

1.2.1.0.6 What Is Docker? **Docker** is a *platform* — a collection of tools — that makes it easy to build, ship, and run containers. Before Docker existed, Linux had all the kernel features needed for containerization (namespaces since 2002, cgroups since 2008), but using them required deep expertise. Docker packaged those features behind a friendly command-line interface, a standard file format for describing containers (the **Dockerfile**), and a public sharing site (**Docker Hub**). That combination is what turned containers from a niche Linux feature into a global industry standard.

Docker as a *company* was founded in 2013. The word "Docker" today may refer to the company, the open-source engine, the CLI tool, or the ecosystem — context tells you which.

1.2.1.0.7 What Is the Docker Engine? **Docker Engine** is the core runtime — the actual software that creates and runs containers on a Linux machine. It has three main pieces:

1. **dockerd** — the Docker daemon. A long-running background process (a "service") that listens for API requests and does the heavy lifting: pulling images, creating containers, managing networks, etc.
2. **Docker CLI (`docker`)** — the command-line tool you type into a terminal. When you run `docker run nginx`, the CLI sends an HTTP request to dockerd, which then does the work.
3. **REST API** — the protocol the CLI uses to talk to the daemon. This same API can be called by any tool, which is how graphical Docker managers, IDE plugins, and CI/CD systems talk to Docker.

1.2.1.0.8 What Is Docker Desktop? **Docker Desktop** is a *packaged application* for Windows and macOS that bundles Docker Engine, the Docker CLI, Docker Compose, Kubernetes, a graphical management UI, and — critically on Windows — a lightweight Linux VM (using WSL2) that provides the Linux kernel needed to run Linux containers. On Linux, you do not need Docker Desktop because your kernel already is Linux; you just install Docker Engine directly.

So the relationship is:

- **Docker Desktop** (the installer you download on Windows) → contains →
- **Docker Engine** (the daemon + CLI) → which uses →
- **containerd** (the low-level container manager) → which uses →
- **runc** (the tiny program that actually calls the Linux kernel to spawn a container)

1.2.1.0.9 Docker Architecture in Detail Docker follows a **client-server** architecture. Here is every piece and how they interact.

The Docker Client is the `docker` command you type. It is dumb — it just formats your command into an HTTP request and sends it to the daemon. The client can talk to a daemon on the same machine (via a Unix socket at `/var/run/docker.sock` on Linux, or a named pipe on Windows) or to a daemon on a remote machine over the network.

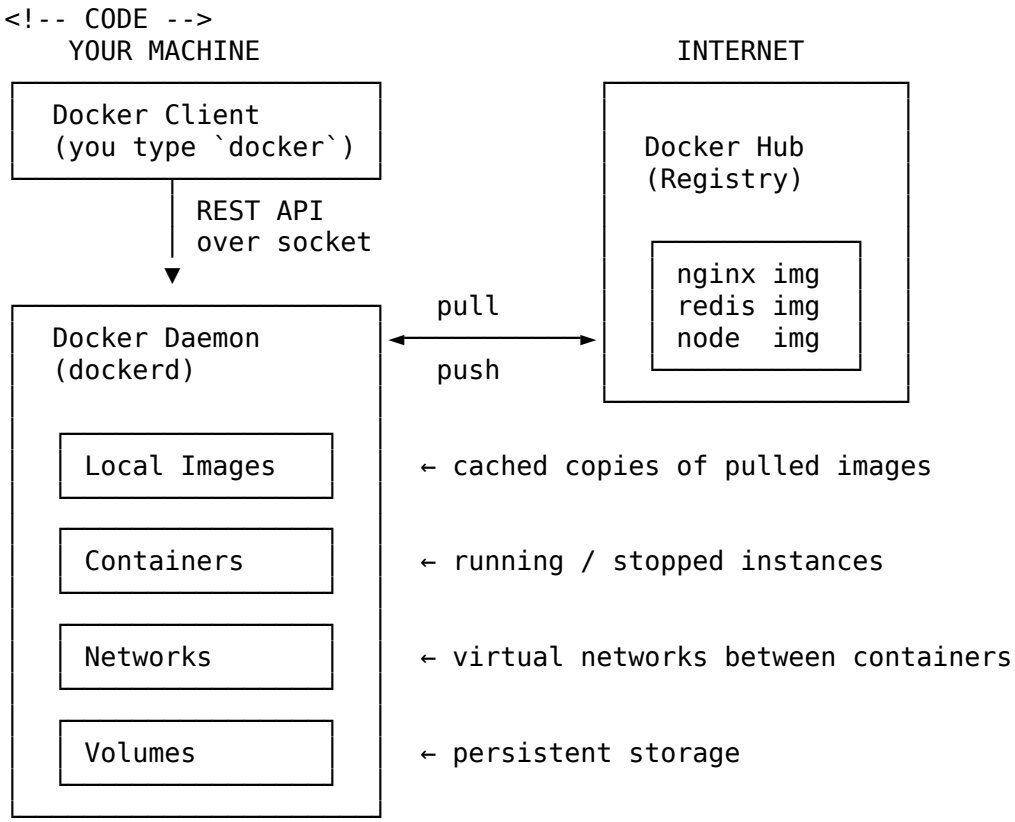
The Docker Daemon (dockerd) is the server. It manages images, containers, networks, and volumes. It pulls images from registries when asked. It creates containers by asking containerd to create them, which asks runc to create them, which asks the Linux kernel.

A Docker Registry is a server that stores Docker images. The default public registry is **Docker Hub** (`hub.docker.com`). Companies also run private registries: AWS ECR, Google Artifact Registry, Azure Container Registry, GitHub Container Registry, or self-hosted registries using the open-source `registry` image.

A Docker Image is a read-only template used to create containers. Think of it as a snapshot of a filesystem plus some metadata (which command to run, which ports are exposed, etc.). Images are built in **layers** — each instruction in a Dockerfile creates one layer, and layers are cached and shared between images to save disk space and download time.

A Docker Container is a running (or stopped) instance of an image. It has its own filesystem (a thin writable layer on top of the read-only image layers), its own network interface, its own process list. You can create many containers from the same image, the same way you can create many objects from the same class in programming.

1.2.1.0.10 Diagram — Docker Client-Daemon-Registry Flow



1.2.1.0.11 Docker vs Virtual Machines — Detailed Comparison

Feature	Docker Container	Virtual Machine
Isolation level	Process-level (kernel namespaces + cgroups)	Hardware-level (hypervisor virtualizes CPU/RAM/disk)
Operating system	Shares host kernel; only user-space is packaged	Full guest OS including its own kernel
Startup time	Milliseconds to a few seconds	Seconds to minutes
Size on disk	Typically 5 MB to a few hundred MB	Typically 1 GB to 20 GB per VM
RAM overhead	Near zero — only what the app uses	Hundreds of MB to GB just for the guest OS
CPU overhead	Near zero — runs as native processes	Small but measurable virtualization overhead
Density (per host)	Hundreds to thousands per server	Dozens per server
Portability	Extremely portable — any host with the same OS family and CPU architecture	Portable across hypervisors of the same family
Cross-OS support	Linux containers need Linux kernel; Windows containers need Windows kernel	Any OS can run on any hypervisor (Linux VMs on Windows host, etc.)

Feature	Docker Container	Virtual Machine
Security isolation	Strong but shares kernel — a kernel exploit breaks isolation	Very strong — hypervisor is a much smaller attack surface
Networking	Virtual bridge networks created by Docker	Virtual switches created by hypervisor
Snapshot/backup	Image commits, volume backups	Full VM snapshots (larger but more complete)
Typical use	Application deployment, microservices, CI/CD	Full OS workloads, legacy apps, security sandboxes
Boot process	No boot — a single process starts	Full BIOS → bootloader → kernel → init → services
Update model	Rebuild image and redeploy container	Patch the guest OS in place, or rebuild image
State	Ephemeral by default; state must be externalized to volumes	Persistent by default; state lives inside the VM

1.2.1.0.12 The Open Container Initiative (OCI) The **Open Container Initiative (OCI)** is a Linux Foundation project founded in 2015 to prevent container fragmentation. It defines two open standards:

1. **OCI Image Specification** — how a container image is packaged, its manifest format, its layer format. Any OCI-compliant tool can read any OCI-compliant image.
2. **OCI Runtime Specification** — how a container is launched from an image, including the JSON config that describes namespaces, cgroups, mount points, etc.

Docker donated its image format and its low-level runtime **runc** to become the reference implementations of these specs. This is critically important because it means images you build with Docker can be run by Podman, containerd, CRI-O, Kubernetes, and any other OCI-compliant runtime — you are not locked into Docker forever. The industry agrees on the format, competes on the tools.

1.2.1.0.13 Brief History of Docker and Containerization

- **1979** — Unix `chroot` introduced. First filesystem isolation primitive.
- **2000** — FreeBSD Jails. First serious multi-facet OS-level virtualization.
- **2002** — Linux namespaces introduced (mount namespace).
- **2005** — Solaris Zones and OpenVZ mature.
- **2008** — cgroups merged into Linux kernel. LXC (Linux Containers) project launches, combining namespaces + cgroups into a usable containerization system.
- **2013** — Solomon Hykes demos Docker at PyCon (originally an internal project at dotCloud, a PaaS company). Docker wraps LXC in a friendly CLI and adds the Dockerfile format and Docker Hub.
- **2014** — Docker replaces LXC with its own `libcontainer` runtime. Google, Red Hat, and others start contributing.
- **2015** — Open Container Initiative (OCI) founded. Docker donates image spec and runc.
- **2015** — Kubernetes 1.0 released by Google. Container orchestration takes off.
- **2016** — Docker Swarm mode integrated into Docker Engine. Docker for Windows and Docker for Mac released.
- **2017** — Docker donates containerd to CNCF. Container ecosystem splits into “low-level” (containerd, runc, CRI-O) and “high-level” (Docker, Podman, Kubernetes) layers.
- **2020** — Kubernetes announces it will deprecate the Docker shim in favor of any CRI-compliant runtime (usually containerd directly). Images and containers remain fully compatible.
- **2021–present** — Docker Desktop becomes a paid product for large enterprises, spurring alternatives like Rancher Desktop, Podman Desktop, and OrbStack. The container ecosystem continues to mature around OCI standards.

1.2.1.0.14 Key Terminology Recap Before moving on, make sure you can define each of these in your own words:

- **Container** — an isolated, running instance of an image.
- **Image** — a read-only template used to create containers.
- **Dockerfile** — a text file with instructions to build an image.
- **Registry** — a server that stores images (e.g., Docker Hub).
- **Docker Engine** — the daemon + CLI on a Linux machine.

- **Docker Desktop** — the installer for Windows/macOS that bundles the engine plus a Linux VM.
- **dockerd** — the Docker daemon process.
- **containerd** — the industry-standard container runtime that Docker uses under the hood.
- **runc** — the low-level tool that actually creates a container from an OCI runtime bundle.
- **Namespace** — Linux kernel feature that isolates what a process can see.
- **cgroup** — Linux kernel feature that limits what a process can use.
- **Layer** — one immutable slice of an image, produced by one Dockerfile instruction.
- **OCI** — the Open Container Initiative, which standardizes image and runtime formats.
- **Hypervisor** — software that runs virtual machines.
- **Virtual Machine (VM)** — a fully virtualized computer with its own guest OS.

Commands

Milestone 1 is theory-only. You will not run commands until Docker is installed in Milestone 2. However, to prepare you for Milestone 2, here are the *conceptual* commands you will meet next — no need to memorize, just recognize:

- `docker version` — shows client and server versions.
- `docker info` — shows detailed daemon state (containers, images, storage driver, etc.).
- `docker run hello-world` — the traditional "am I working" test.

Command Options/Flags

Because no commands are executed in this milestone, there are no flags to detail here. Milestone 2 will introduce the first real flag tables.

Examples

The following are **conceptual examples** — do not run them yet. They illustrate what Docker commands look like so the syntax is not a surprise in Milestone 2.

Example 1 — Checking whether Docker is installed:

```
docker version
```

Example 2 — Running your first container:

```
docker run hello-world
```

Example 3 — Listing running containers:

```
docker ps
```

Command Exercises

Because this milestone is theory-only, these exercises test *conceptual understanding* rather than command execution. Answer each in your head or on paper, then reveal the solution.

1.2.1.0.15 Exercise 1.1 — Matching: Component to Role Match each Docker component (left) to its role (right):

Component	Role
A. dockerd	1. A snapshot filesystem used to create containers
B. Docker CLI	2. A public server that stores images
C. Docker Hub	3. A background service that manages containers
D. Image	4. A running instance of an image
E. Container	5. The command-line tool you type into

 Solution

```
A → 3  (dockerd is the background daemon)
B → 5  (Docker CLI is the command-line tool)
C → 2  (Docker Hub is the public registry)
D → 1  (An Image is the read-only template / snapshot)
E → 4  (A Container is a running instance of an image)
```

1.2.1.0.16 Exercise 1.2 — Predict the Output: VM vs Container You are told that starting a virtual machine takes 60 seconds and uses 2 GB of RAM just for its guest OS. Starting an equivalent container of nginx takes about 0.5 seconds and uses about 10 MB of RAM. If a physical server has 64 GB of RAM, roughly how many idle nginx VMs versus how many idle nginx containers could you run on it (ignoring the host OS overhead)?

 Solution

```
VMs   : 64 GB / 2 GB   =  ~32 VMs
Containers: 64 GB / 10 MB = ~6,400 containers

This ~200x density difference is exactly why the industry
moved from VMs to containers for stateless workloads.
```

1.2.1.0.17 Exercise 1.3 — Troubleshooting Thought Experiment A junior developer says, "I built a Docker image for my Windows desktop application on my Linux server, but when I try to run it on a Windows machine with Docker Desktop, it fails." Based on what you learned about the shared kernel model, explain in one sentence why this fails, and then in one sentence what needs to change.

 Solution

```
WHY IT FAILS:
Linux containers share the host's Linux kernel, so a Linux-based
image cannot execute Windows-native code — and vice versa.

WHAT NEEDS TO CHANGE:
Either package the app as a Windows container (built on a Windows
base image, run on Windows Server or Docker Desktop in Windows
container mode), OR rewrite the app as a Linux-compatible program
so it can run in a Linux container.
```

Hands-On Assignment

Task: Produce a one-page "cheat sheet" (in Markdown or on paper) that answers the following five questions in your own words. Do not copy from this document — restating in your own words is what cements understanding.

1. What problem does Docker solve, in one sentence?
2. Explain the difference between a **Virtual Machine** and a **Container** using an analogy that is *not* apartments/houses (invent your own).
3. Draw (ASCII or on paper) the flow from typing `docker run nginx` on your keyboard to nginx actually running. Label at least: Client, Daemon, Registry, Image, Container.
4. Define these five terms in one sentence each: **Image**, **Container**, **Registry**, **Dockerfile**, **Layer**.
5. Name three real-world scenarios where you would pick a Virtual Machine over a container.

Acceptance criteria:

- ☐ All five questions answered in *your own words*, not copied.
- ☐ Analogy in question 2 is genuinely different from the apartment/house one.
- ☐ Diagram in question 3 includes all five labeled components.
- ☐ Definitions in question 4 are one sentence each and technically correct.
- ☐ Three VM scenarios in question 5 are distinct (not three variations of "legacy Windows app").

Mini-Project

1.2.1.0.18 Project Title Docker Architecture Visual Reference Guide

1.2.1.0.19 Objective You will create a single Markdown document (`docker-architecture-guide.md`) that will serve as your personal reference for the rest of this course. Because Milestone 1 is pure theory, the deliverable is a document — but a *good* one, because you will look back at it constantly as you learn commands in later milestones and want a refresher on the mental model.

1.2.1.0.20 Requirements Your Markdown document must contain, in this order:

1. **A title and one-paragraph introduction** stating who the guide is for (you, in three months, when you have forgotten some of this) and what problem containers solve.

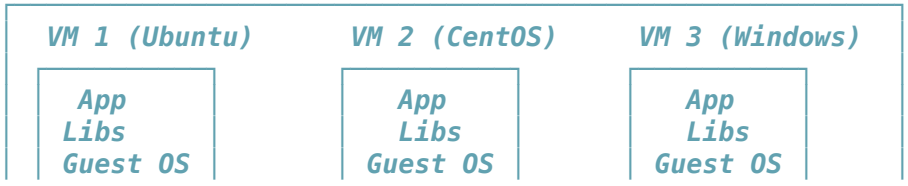
2. **An ASCII diagram of Virtual Machine architecture** with at least five labeled layers (Hardware, Hypervisor, Guest OS, Libraries, App).
3. **An ASCII diagram of Container architecture** with at least four labeled layers (Hardware, Host OS/Kernel, Docker Engine, Container = App + Libraries).
4. **A labeled Docker architecture diagram** showing the flow: Docker Client → Docker Daemon → Registry (Docker Hub) → Local Images → Running Containers. Every arrow must be labeled with what flows through it (e.g., "REST API request", "pull image", "create container").
5. **A comparison table** with at least eight rows comparing VMs and Containers on: isolation, startup time, size, RAM overhead, portability, cross-OS support, density, and typical use.
6. **A decision matrix** — a table titled "When to Use Containers vs VMs" with at least six scenarios and a recommendation column ("Container", "VM", or "Either — explain").
7. **A glossary** of at least 15 terms with one-sentence definitions. Include at minimum: Container, Image, Dockerfile, Registry, Docker Hub, Docker Engine, Docker Desktop, dockerd, containerd, runc, namespace, cgroup, layer, OCI, hypervisor, virtual machine.
8. **A "history timeline"** listing at least six milestones in the evolution of containers, from chroot (1979) to today.

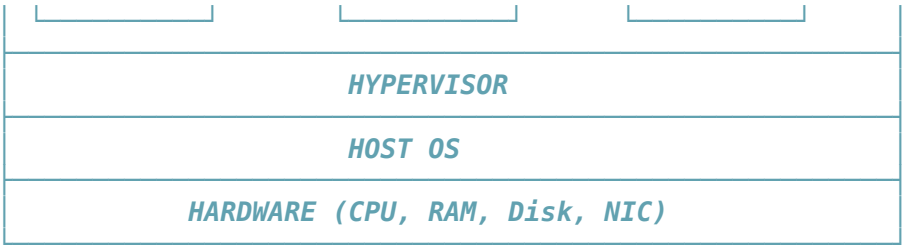
1.2.1.0.21 □ Step-by-Step Guidance

1. Create a new folder called `docker-mastery` on your Desktop. Inside it, create a file called `docker-architecture-guide.md`.
2. Open the file in VS Code or any Markdown editor with preview support.
3. Write the introduction paragraph first — this forces you to articulate the "why" before the "what".
4. Sketch the two architecture diagrams on paper first, then convert them to ASCII using box-drawing characters (`┌ ┐ └ ┘ ─ │ ├ ┤`). Keep them simple.
5. Build the client-daemon-registry diagram next. Label every arrow.
6. Draft the comparison table. Try to write each cell without looking back at this milestone — check afterward.
7. For the decision matrix, brainstorm six *distinct* scenarios (e.g., "run a legacy Windows Server 2008 app", "run 500 microservices on one cluster", "sandbox untrusted user code with maximum isolation"). Assign each a recommendation and a one-sentence justification.
8. Write the glossary last, when the terms are freshest.
9. End with the timeline.
10. Save the file. Commit to reviewing it at the end of every subsequent milestone to reinforce the mental model.

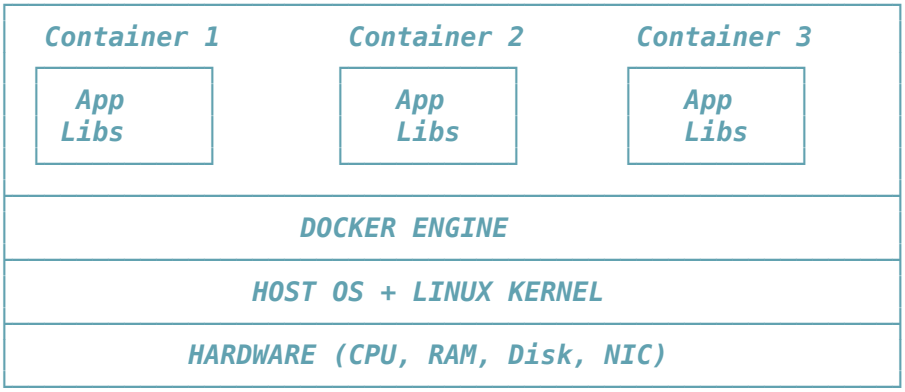
1.2.1.0.22 □ Complete Mini-Project Solution □ Complete Mini-Project Solution

```
# Docker Architecture Visual Reference Guide

## Introduction

This guide is for **future me**, three months from now, when I have written
a hundred Dockerfiles and forgotten *why* containers work the way they do.
It captures the mental model of Docker: what a container really is, how it
differs from a virtual machine, and how the pieces of Docker fit together.

The core problem Docker solves is environmental drift — the fact that
software behaves differently on different machines because of hidden
differences in OS libraries, runtime versions, and configuration.
Docker packages an application together with its entire user-space
environment into a portable unit called a container, so "works on my
machine" becomes "works on every machine".

## 1. Virtual Machine Architecture

```

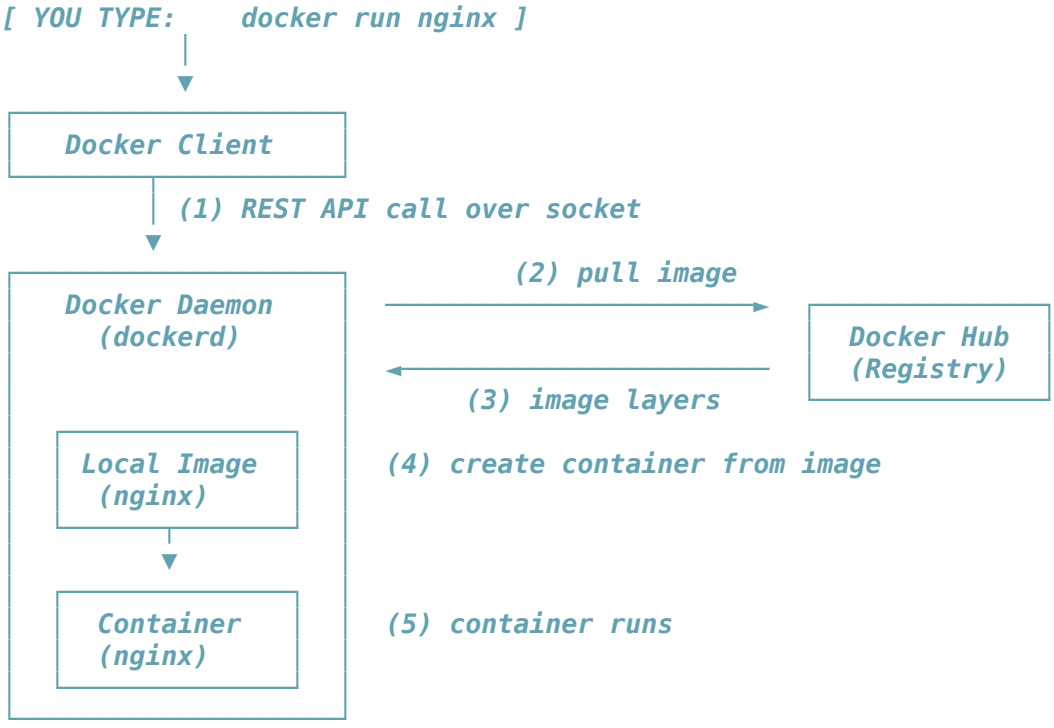





```

## 2. Container Architecture

```




```

## 3. Docker Client-Daemon-Registry Flow

```




```

## 4. Comparison Table

| Feature | Container | Virtual Machine |
|---|---|---|
| Isolation | Namespaces/cgroups | Hypervisor |
| Startup time | ms – seconds | Seconds – minutes |
| Size on disk | 5 MB – few 100 MB | 1 GB – 20 GB |
```

RAM overhead	~0	Hundreds of MB – GB
Portability	Very high (OCI)	High (per hypervisor)
Cross-OS	Same kernel family only	Any guest on any host
Density per server	100s – 1000s	10s
Typical use	Microservices	Full OS workloads

5. When to Use Containers vs VMs – Decision Matrix

Scenario	Choose	Why
Deploy 200 stateless microservices	Container	Density and fast roll
Run legacy Windows Server 2008 application	VM	Container needs match
Sandbox untrusted user code with maximum isolation	VM	Hypervisor is a small
Ephemeral CI/CD build agents that spin up per commit	Container	Startup speed and ima
Multi-tenant SaaS: strong customer isolation	VM	Kernel exploit doesn't
Local dev environment mirroring production	Container	Same image dev → CI →

6. Glossary

- **Container** – an isolated running instance of an image.
- **Image** – a read-only template used to create containers.
- **Dockerfile** – a text file with instructions to build an image.
- **Registry** – a server that stores and distributes images.
- **Docker Hub** – the default public registry.
- **Docker Engine** – the daemon + CLI on a Linux machine.
- **Docker Desktop** – Windows/Mac installer bundling Engine + a Linux VM.
- **dockerd** – the Docker daemon (background service).
- **containerd** – the OCI-compliant runtime Docker uses under the hood.
- **runc** – the low-level tool that spawns a container from an OCI bundle.
- **Namespace** – kernel feature isolating what a process can see.
- **cgroup** – kernel feature limiting what a process can use.
- **Layer** – one immutable slice of an image built by one Dockerfile step.
- **OCI** – Open Container Initiative, industry standard for images/runtimes.
- **Hypervisor** – software that runs virtual machines.
- **Virtual Machine (VM)** – a full virtualized computer with its own kernel.

7. History Timeline

- **1979** – Unix `chroot` – first filesystem isolation.
- **2000** – FreeBSD Jails – early multi-facet OS-level isolation.
- **2008** – cgroups merged into Linux kernel; LXC launches.
- **2013** – Docker publicly launched at PyCon.
- **2015** – OCI founded; Docker donates image spec and runc.
- **2015** – Kubernetes 1.0 released.
- **2017** – Docker donates containerd to CNCF.
- **2020+** – Kubernetes standardizes on containerd; ecosystem matures.

1.2.1.0.23 ✅ Verification Checklist

- ☐ Document is saved as `docker-architecture-guide.md` in your `docker-mastery` folder.
- ☐ Introduction paragraph explains, in your own words, the problem containers solve.
- ☐ VM architecture ASCII diagram is present with at least five labeled layers.
- ☐ Container architecture ASCII diagram is present with at least four labeled layers.
- ☐ Docker client-daemon-registry diagram shows five components with labeled arrows.
- ☐ Comparison table has at least eight rows.
- ☐ Decision matrix has at least six distinct scenarios with recommendations.
- ☐ Glossary has at least 15 terms — all listed in the requirements are included.
- ☐ History timeline has at least six milestones.
- ☐ You can explain every diagram out loud to an imaginary junior developer without looking at the notes.

1.2.1.0.24 🚀 Bonus Challenges

1. **Compare three container runtimes.** Extend the guide with a short section comparing Docker, Podman, and containerd — what each is, how they relate, and when someone might choose one over another.
2. **Add a "Docker on Windows" architecture diagram.** Show explicitly how Docker Desktop uses WSL2 as the Linux kernel provider. Label the Windows host, WSL2 VM, dockerd inside WSL2, and containers inside dockerd.
3. **Write a 200-word "elevator pitch"** — imagine explaining containers to your non-technical grandparent in about 60 seconds. Add it to the top of the guide as a "What I would tell a non-engineer" callout.

Scenario (Real-World Use Case)

You are a new engineer at a mid-sized fintech company. On day one, your manager tells you: "Our monolithic Java app runs on 40 VMs in AWS. We spend around $18,000 a month on those VMs. Each VM sits at 8% CPU utilization on average — most of the RAM and CPU is wasted on the guest OS. Leadership wants to know if we can cut that bill in half by migrating to containers."

Your job on day one is not to actually migrate anything — it is to explain, in one clear meeting to non-engineering executives, *why* containers would even help. You draw a whiteboard diagram of VM architecture next to container architecture, walk through the shared-kernel concept, show the density comparison table, and highlight that the same physical hardware could host ~10x more containers than VMs because containers do not carry a full guest OS each.

The executives green-light a proof-of-concept. Because you understood the fundamentals of Milestone 1, you now know what problem you are actually solving in the next 19 milestones: turning that monolith into portable, dense, fast-starting containers that can be orchestrated across a much smaller fleet of hosts.

This exact scenario — "why containers?" — is the first conversation of every containerization project in the industry. If you can have it clearly, you are already ahead of most junior engineers.

Checkpoint Quiz

Question 1. Which of the following does a container share with its host?

A. Its own dedicated CPU
B. The host's Linux kernel
C. Its own hypervisor
D. Its own BIOS

Click to reveal answer

B. The host's Linux kernel. Containers share the host kernel, which is exactly why they are so lightweight and why Linux containers cannot natively run on a Windows kernel.

Question 2. True or false: A Docker image is a running instance of a container.

Click to reveal answer

False. The relationship is the reverse: a Docker **container** is a running instance of an **image**. An image is a read-only template; a container is a live process created from that template.

Question 3. Which two Linux kernel features are the foundation of container isolation?

Click to reveal answer

Namespaces (which isolate what a process can *see* — e.g., its own PID list, its own network stack, its own filesystem view) and **cgroups** (which isolate what a process can *use* — e.g., how much CPU, memory, or I/O it can consume).

Question 4. What is the difference between Docker Engine and Docker Desktop?

Click to reveal answer

Docker Engine is the core daemon + CLI that actually runs containers on a Linux machine. **Docker Desktop** is a Windows/Mac desktop application that bundles Docker Engine together with a lightweight Linux VM (via WSL2 on Windows or a hypervisor on Mac), a graphical UI, Kubernetes, and Docker Compose. On Linux, you install Docker Engine directly; on Windows and Mac, you install Docker Desktop which contains Docker Engine.

Question 5. Why does the Open Container Initiative (OCI) matter to a developer who only ever uses Docker?

Click to reveal answer

Because OCI defines the standard image and runtime formats, an image built with Docker can be run by Podman, containerd, CRI-O, or any Kubernetes cluster — with no changes. This protects the developer from vendor lock-in: the skills, images, and Dockerfiles transfer to any OCI-compliant tool. Even if Docker (the company) disappeared tomorrow, your images would still run everywhere.

⬆ Back to Table of Contents

1.2.2 Milestone 2: Installing Docker on Windows

Theory

Before you can run a single container, Docker must be installed and healthy on your Windows machine. This milestone walks you from a completely fresh Windows laptop to a working Docker Desktop install with your first container running. Do not skim — a broken install is the #1 reason beginners quit Docker.

1.2.2.0.1 The Windows-Specific Challenge Recall from Milestone 1 that Linux containers require a Linux kernel. Windows does not have a Linux kernel — it has the Windows NT kernel. So how does Docker Desktop run Linux containers on Windows? Answer: it uses the **Windows Subsystem for Linux version 2 (WSL2)**, a Microsoft-provided lightweight Linux VM that ships with Windows 10/11 and provides a real Linux kernel to programs running on Windows. Docker Desktop installs its Linux daemon (`dockerd`) inside WSL2 and exposes it to your Windows command prompt through a socket. From your point of view, you type `docker run nginx` in PowerShell and it "just works" — under the hood, Windows PowerShell is talking to a Linux daemon inside a Linux VM.

There is also an older option — running Docker on the **Hyper-V** hypervisor — but WSL2 is faster, uses less RAM, and is the recommended default on modern Windows. This milestone assumes WSL2.

1.2.2.0.2 System Requirements To install Docker Desktop on Windows with WSL2, you need:

- **Windows version:** Windows 10 64-bit version 22H2 (Home, Pro, Enterprise, or Education), or Windows 11 64-bit (Home, Pro, Enterprise, or Education). Older versions are not supported.
- **CPU:** 64-bit processor with **Second Level Address Translation (SLAT)**. Every Intel Core i-series CPU from 2010 onward has this, as does every AMD Ryzen and modern AMD FX chip. If your CPU is more than about 12 years old, check.
- **CPU virtualization:** must be **enabled in BIOS/UEFI**. On Intel this is called "VT-x" or "Intel Virtualization Technology"; on AMD it is called "AMD-V" or "SVM Mode". This is turned on in most business laptops by default but often off on consumer/gaming PCs.
- **RAM:** minimum 4 GB, recommended 8 GB or more. Docker Desktop uses ~2 GB just to run the WSL2 VM and services.
- **Disk:** at least 10 GB free for Docker Desktop plus space for images and containers.
- **WSL2 kernel:** Docker Desktop's installer will prompt you to install the WSL2 Linux kernel update if it is missing.

1.2.2.0.3 What Gets Installed When you run the Docker Desktop installer on Windows, it installs:

1. **The Docker Desktop application** — a graphical management UI in your Start menu.
2. **The Docker CLI (`docker`)** — added to your Windows PATH so it runs from PowerShell or Command Prompt.
3. **Docker Compose (`docker compose`)** — bundled as a plugin (no separate install).
4. **A WSL2 distribution called `docker-desktop`** — a minimal Linux VM containing `dockerd` and `containerd`.
5. **A WSL2 distribution called `docker-desktop-data`** (older versions) — a separate VM for image and volume storage. In current versions, this may be a single distro.
6. **A local Kubernetes cluster (optional)** — you can turn on Kubernetes in Docker Desktop settings.
7. **The Docker Scout, Docker Extensions, and other tools** — accessible from the Docker Desktop UI.

Commands

The commands introduced in this milestone are:

- `wsl --status` — check WSL version and default distro.

- `wsl --list --verbose` — list installed WSL distributions.
- `wsl --install` — install WSL and Ubuntu (first-time setup).
- `wsl --set-default-version 2` — make WSL2 the default.
- `wsl --update` — update the WSL kernel.
- `systeminfo` — show Windows version and virtualization support (PowerShell).
- `docker version` — show client and server version details.
- `docker info` — show detailed daemon state.
- `docker system info` — alias of `docker info`.
- `docker run hello-world` — run the canonical first-container test.

Command Options/Flags

1.2.2.0.4 `docker version` flags

Flag	Short Form	Description	Example
`--format`	`-f`	Format output using a Go template	`docker version --format '{{.Server.Version}}'`
`--help`	—	Show help for the command	`docker version --help`

1.2.2.0.5 `docker info` flags

Flag	Short Form	Description	Example
`--format`	`-f`	Format output using a Go template or `json`	`docker info --format json`
`--help`	—	Show help for the command	`docker info --help`

1.2.2.0.6 `docker run` flags (introduced here; complete table in Milestone 3)

Flag	Short Form	Description	Example
`--rm`	—	Remove container automatically when it exits	`docker run --rm hello-world`

1.2.2.0.7 `wsl` flags (Windows PowerShell)

Flag	Short Form	Description	Example
`--status`	—	Show WSL configuration	`wsl --status`
`--list`	`-l`	List installed distributions	`wsl -l -v`
`--verbose`	`-v`	Verbose output (with `--list`)	`wsl --list --verbose`
`--install`	—	Install WSL and default Linux distro	`wsl --install`
`--set-default-version`	—	Set default WSL version (1 or 2)	`wsl --set-default-version 2`
`--update`	—	Update the WSL Linux kernel	`wsl --update`
`--shutdown`	—	Shut down all running WSL VMs	`wsl --shutdown`

Examples

1.2.2.0.8 Step 1 — Check your Windows version

Open PowerShell as Administrator (right-click Start → "Terminal (Admin)" or "Windows PowerShell (Admin)") and run:

```
winver
```

A dialog appears showing your Windows edition and version. You need Windows 10 22H2+ or Windows 11.

Alternatively, from PowerShell:

```
systeminfo | Select-String "OS Name","OS Version"
```

```
OS Name:                   Microsoft Windows 11 Pro
OS Version:                10.0.22631 N/A Build 22631
```

1.2.2.0.9 Step 2 — Verify CPU virtualization is enabled

```
systeminfo | Select-String "Hyper-V"
```

```
Hyper-V Requirements:      VM Monitor Mode Extensions: Yes
                           Virtualization Enabled In Firmware: Yes
                           Second Level Address Translation: Yes
                           Data Execution Prevention Available: Yes
```

All four must be Yes. If "Virtualization Enabled In Firmware" is No, you must reboot into your BIOS/UEFI and enable VT-x (Intel) or SVM (AMD).

1.2.2.0.10 Step 3 — Install and enable WSL2

```
wsl --install
```

```
Installing: Virtual Machine Platform
Virtual Machine Platform has been installed.
Installing: Windows Subsystem for Linux
Windows Subsystem for Linux has been installed.
Installing: Ubuntu
Ubuntu has been installed.
The requested operation is successful. Changes will not be effective
until the system is rebooted.
```

Reboot when prompted. After reboot:

```
wsl --set-default-version 2
wsl --update
wsl --status
```

```
Default Distribution: Ubuntu
Default Version: 2

Windows Subsystem for Linux was last updated on 2025-10-15
The WSL kernel version is: 5.15.167.4-1
```

1.2.2.0.11 Step 4 — Download and install Docker Desktop

1. Go to **https://www.docker.com/products/docker-desktop/**
2. Click **"Download for Windows"** → **"Docker Desktop Installer.exe"** (about 700 MB).
3. Double-click the installer.
4. On the "Configuration" screen, leave **"Use WSL 2 instead of Hyper-V"** checked (this is the default).
5. Click **OK** and wait 5–10 minutes.
6. When installation completes, click **"Close and log out"** — you must sign out of Windows for group membership changes to take effect.
7. Sign back in. Docker Desktop icon (a whale with containers on its back) will appear in the system tray.
8. Docker Desktop launches; accept the license agreement; skip or complete the sign-in prompt; wait for the whale icon to stop animating.

1.2.2.0.12 Step 5 — Verify installation with `docker version`

Open a new PowerShell window and run:

```
docker version
```

```
Client:
 Cloud integration: v1.0.35+desktop.16
 Version:           27.3.1
 API version:       1.47
 Go version:        go1.22.7
 Git commit:        ce12230
 Built:             Fri Sep 20 11:41:00 2024
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.35.0 (172550)
 Engine:
  Version:          27.3.1
  API version:      1.47 (minimum version 1.24)
  Go version:       go1.22.7
  Git commit:       41ca978
  Built:            Fri Sep 20 11:41:00 2024
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.7.21
  GitCommit:        472731909fa34bd7bc9c087e4c27943f9835f111
 runc:
  Version:          1.1.13
  GitCommit:        v1.1.13-0-g58aa920
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Explaining every line:

- **`Client:`** — this section describes the `docker` CLI you just typed on your Windows machine.
  - `Cloud integration` — plugin for Docker Cloud features.
  - `Version` — the client version (`27.3.1`). Should match or be compatible with the server.
  - `API version` — the version of the Docker REST API the client speaks.
  - `Go version` — Docker is written in Go; this is the compiler version used to build it.
  - `Git commit` — the exact git commit hash Docker was built from (useful for bug reports).
  - `Built` — timestamp of the build.
  - `OS/Arch` — `windows/amd64` means the client binary is a Windows 64-bit executable.
  - `Context` — `desktop-linux` means the CLI is currently talking to the Linux daemon inside Docker Desktop. Other contexts might point to remote Docker hosts.
- **`Server:`** — this section describes `dockerd` running inside WSL2.
  - `Docker Desktop 4.35.0 (172550)` — Docker Desktop application version.
  - `Engine.Version` — the daemon version.
  - `Engine.API version` — API version the daemon speaks; `minimum version 1.24` means older clients back to that API version still work.
  - `Engine.OS/Arch` — `linux/amd64` — confirms the daemon runs on a Linux kernel inside WSL2.
  - `Engine.Experimental: false` — no experimental features enabled.
- **`containerd:`** — the container runtime Docker uses under the hood (industry-standard, donated to CNCF).
- **`runc:`** — the low-level tool that actually creates containers via kernel syscalls.
- **`docker-init:`** — a tiny "init" process that runs as PID 1 inside each container (only when you pass `--init`), reaping zombie processes.

If you see `Server:` details, congratulations — the daemon is running. If you see `error during connect: this error may indicate that the docker daemon is not running`, Docker Desktop hasn't finished starting up. Wait 30 seconds and try again.

1.2.2.0.13 Step 6 — Verify installation with `docker info`

```
docker info
```

```
Client:
 Version:    27.3.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.) v0.17.1-desktop.1
  compose: Docker Compose (Docker Inc.) v2.29.7-desktop.1
  ...

Server:
 Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
 Images: 0
 Server Version: 27.3.1
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version: 472731909fa34bd7bc9c087e4c27943f9835f111
 runc version: v1.1.13-0-g58aa920
 init version: de40ad0
 Security Options:
  seccomp
   Profile: unconfined
  cgroupns
 Kernel Version: 5.15.167.4-microsoft-standard-WSL2
 Operating System: Docker Desktop
 OSType: linux
 Architecture: x86_64
 CPUs: 8
 Total Memory: 7.663GiB
 Name: docker-desktop
 ID: 5a08b7c3-....
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 HTTP Proxy: http.docker.internal:3128
 HTTPS Proxy: http.docker.internal:3128
 No Proxy: hubproxy.docker.internal
 Labels:
 Experimental: false
 Insecure Registries:
  hubproxy.docker.internal:5555
  127.0.0.0/8
 Live Restore Enabled: false
```

Explaining every line:

- **`Containers`** — how many containers exist locally (running + paused + stopped).
- **`Images`** — how many images are cached locally.

- **`Server Version`** — daemon version (should match client where possible).
- **`Storage Driver: overlay2`** — the filesystem driver Docker uses to stack image layers. `overlay2` is the modern default.
- **`Backing Filesystem: extfs`** — the underlying filesystem beneath overlay2 (ext4 inside WSL2).
- **`Native Overlay Diff: true`** — kernel supports native diffs for overlay2 (faster builds).
- **`Logging Driver: json-file`** — how container stdout/stderr is captured to disk. Default is JSON files under `/var/lib/docker/containers/`.
- **`Cgroup Driver: cgroupfs`** — how Docker talks to cgroups. Alternative is `systemd`.
- **`Cgroup Version: 2`** — cgroup v2 is the modern unified hierarchy.
- **`Plugins.Volume/Network/Log`** — built-in drivers for each area.
- **`Swarm: inactive`** — Docker Swarm cluster mode is not initialized (that's Milestone 15).
- **`Runtimes`** — available container runtimes. Default is `runc`.
- **`Security Options.seccomp`** — a Linux syscall-filtering mechanism used to sandbox containers.
- **`Kernel Version`** — the Linux kernel version inside WSL2. Notice `microsoft-standard-WSL2` in the name.
- **`Operating System: Docker Desktop`** — signals this daemon is running under Docker Desktop's minimal Linux.
- **`OSType: linux`** — this daemon runs Linux containers.
- **`Architecture: x86_64`** — 64-bit Intel/AMD.
- **`CPUs`** — how many CPU cores are allocated to the WSL2 VM (default is often all cores).
- **`Total Memory`** — RAM allocated to WSL2 (adjustable in Docker Desktop settings).
- **`Docker Root Dir`** — where Docker stores images, containers, volumes (inside WSL2).
- **`Live Restore Enabled: false`** — if `true`, containers keep running when the daemon restarts.

1.2.2.0.14 Step 7 — Run your first container

```
docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:e2fc4e5012d16e7fe466f5291c476431beaa1f9b90a5c2125b493ed28e2aba57
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Explaining every line:

- `Unable to find image 'hello-world:latest' locally` — Docker checked its local image cache; the image was not there.
- `latest: Pulling from library/hello-world` — Docker is downloading the `latest` tag of the `hello-world` image from the `library` (official images) namespace on Docker Hub.
- `c1ec31eb5944: Pull complete` — one layer downloaded and unpacked. `hello-world` has just one tiny layer.
- `Digest: sha256:e2fc...` — the cryptographic hash of the image. This is a content-addressable identifier — if two images have the same digest, they are bit-for-bit identical.

- `Status: Downloaded newer image for hello-world:latest` — cache updated.
- `Hello from Docker! ... To generate this message, Docker took the following steps:` — the container itself is a tiny program that prints this text.
- Steps 1–4 in the output describe *exactly* the Client → Daemon → Registry → Container flow you learned in Milestone 1. Read them and match each step to your mental model.

If you see this output, **your Docker install is 100% functional**. You have completed the hardest part of learning Docker.

1.2.2.0.15 Configuring Docker Desktop Settings Open Docker Desktop from the system tray → click the ⚙ (gear) icon → Settings.

- **General** — auto-start, telemetry, use containerd for image store (experimental).
- **Resources → Advanced** — set CPU cores, memory (RAM), swap, and disk image size allocated to WSL2. For most laptops, 4 CPUs and 4–6 GB RAM is a good starting point.
- **Resources → WSL Integration** — enable the toggle for each installed WSL distro (like Ubuntu) so you can also use the `docker` CLI from inside `wsl -d Ubuntu`.
- **Docker Engine** — a JSON editor for `daemon.json`. This is the daemon configuration file. Example customizations:

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
      "enabled": true
    }
  },
  "experimental": false,
  "features": {
    "buildkit": true
  },
  "insecure-registries": [],
  "registry-mirrors": []
}
```

After editing, click **Apply & Restart** — the daemon reloads.

- **Kubernetes** — enable a single-node Kubernetes cluster (adds ~1 GB RAM).
- **Software Updates** — check for new Docker Desktop releases.

1.2.2.0.16 Troubleshooting

- **“WSL 2 installation is incomplete”** — Run `wsl --update` in an admin PowerShell, reboot, and restart Docker Desktop.
- **“Virtualization is disabled”** — Reboot into BIOS/UEFI, enable VT-x (Intel) or SVM (AMD), save and exit.
- **“Hardware assisted virtualization and data execution protection must be enabled in the BIOS”** — Same as above.
- **Hyper-V conflicts** (e.g., you use VirtualBox and Docker Desktop won’t start) — Older VirtualBox versions conflicted with Hyper-V. Update VirtualBox to 6.1+ and update Windows, or run only one hypervisor at a time.
- **“Docker Desktop is starting…” forever** — Restart Docker Desktop from the tray icon → Troubleshoot → Restart. If that fails, Troubleshoot → Reset to factory defaults (destroys all containers and images).
- **“docker: command not found”** in PowerShell — Docker Desktop failed to add itself to your PATH. Sign out and back in, or reboot.
- **`docker version` shows client but no server** — the daemon has not finished starting; wait or restart Docker Desktop.
- **“error during connect: … The system cannot find the file specified”** — the WSL2 distro `docker-desktop` is stopped. Restart Docker Desktop, or `wsl --shutdown` then relaunch.

Command Exercises

1.2.2.0.17 Exercise 2.1 — `docker version` basic Run `docker version` and identify (a) your client version, (b) your server version, (c) whether they match, and (d) the OS/Arch of the server.

 Solution

```
docker version
```

```
Client:
 Version:  27.3.1
 OS/Arch:  windows/amd64

Server: Docker Desktop 4.35.0
 Engine:
  Version:  27.3.1
  OS/Arch:  linux/amd64
```

Answers: (a) Client version: 27.3.1 (yours may differ) (b) Server version: 27.3.1 (c) They match (client and server are the same major.minor) (d) Server OS/Arch: linux/amd64 — the daemon runs on Linux inside WSL2, even though you typed the command in Windows.

1.2.2.0.18 Exercise 2.2 — `docker version` with format flag Use the `--format` flag with a Go template to display *only* the server version number, nothing else.

 Solution

```
docker version --format '{{.Server.Version}}'
```

```
27.3.1
```

This is useful in scripts — you can capture just the version into a variable without parsing full text.

1.2.2.0.19 Exercise 2.3 — `docker version` troubleshooting Simulate a broken daemon: from Docker Desktop tray icon, click “Quit Docker Desktop”. Now run `docker version` and observe the difference. Then restart Docker Desktop and confirm it recovers.

 Solution

```
docker version
```

```
Client:
 Version:  27.3.1
 ...
error during connect: Get "http://%2F%2F.%2Fpipe%2Fdocker_engine/v1.47/version":
open //./pipe/docker_engine: The system cannot find the file specified.
```

Diagnosis: the CLI (client) still works — it printed the client block — but it cannot reach the daemon over the named pipe because Docker Desktop is stopped. Fix:

1. Launch Docker Desktop from the Start menu.
2. Wait until the tray icon stops animating.
3. Re-run `docker version` — the Server block reappears.

1.2.2.0.20 Exercise 2.4 — `docker info` basic Run `docker info` and answer: (a) how many images and containers exist, (b) what is the storage driver, (c) what is the total memory allocated to the daemon.

 Solution

```
docker info
```

```
Server:
 Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
 Images: 1
 Storage Driver: overlay2
 Total Memory: 7.663GiB
```

Answers: (a) 1 container (stopped) and 1 image (from hello-world) (b) Storage Driver: overlay2 (c) Total Memory: 7.663 GiB (yours may differ; adjustable in Docker Desktop → Settings → Resources)

1.2.2.0.21 Exercise 2.5 — `docker info` with JSON format Output `docker info` as JSON and identify the field name for the kernel version.

 Solution

```
docker info --format json
```

```
{
  "ID": "5a08b7c3-...",
  "Containers": 1,
  "Images": 1,
  "ServerVersion": "27.3.1",
  "KernelVersion": "5.15.167.4-microsoft-standard-WSL2",
  "OperatingSystem": "Docker Desktop",
  "OSType": "linux",
  "Architecture": "x86_64",
  ...
}
```

The field name is `KernelVersion`. In scripts you could extract it with:

```
docker info --format '{{.KernelVersion}}'
```

1.2.2.0.22 Exercise 2.6 — `docker info` filtered output Use `docker info` with a Go template to print only three lines: `CPUs`, `Total Memory`, and `Kernel Version`, each on its own line.

 Solution

```
docker info --format 'CPUs: {{.NCPU}}{{"\n"}}Memory: {{.MemTotal}}{{"\n"}}Kernel: {{.KernelV
```

```
CPUs: 8
Memory: 8225849344
Kernel: 5.15.167.4-microsoft-standard-WSL2
```

Note: `MemTotal` is in bytes here (not the human-readable "GiB" you see in plain `docker info` output).

1.2.2.0.23 Exercise 2.7 — `docker system info` basic Confirm that `docker system info` is an alias for `docker info` by comparing their output.

 Solution

```
docker system info | Out-File info1.txt
docker info | Out-File info2.txt
Compare-Object (Get-Content info1.txt) (Get-Content info2.txt)
```

```
(no output — files are identical)
```

Confirmed. `docker system info` and `docker info` produce identical output. `docker system` is the "system-management" command group, and `info` is one of its sub-commands (others include `df`, `prune`, `events`).

1.2.2.0.24 Exercise 2.8 — Explore Docker Desktop GUI Open Docker Desktop → Settings → Resources → Advanced. Note current CPU, Memory, and Swap. Change memory to 4 GB, click Apply & Restart, then verify with `docker info` that `Total Memory` reflects the change.

 Solution

```
Steps:
1. Docker Desktop tray icon → Settings → Resources → Advanced
2. Note current values (e.g., Memory: 8 GB, CPUs: 8, Swap: 1 GB)
3. Drag Memory slider to 4 GB
4. Click "Apply & Restart"
5. Wait ~30 seconds for daemon to restart
6. Run:
```

```
docker info --format '{{.MemTotal}}'
```

```
4114677760
```

4114677760 bytes ≈ 3.83 GiB (WSL2 reserves a bit of overhead, so the reported total is slightly less than the 4 GB you allocated). Change memory back to your original value when done.

Hands-On Assignment

Task: From a fresh terminal, prove your Docker installation is working end-to-end.

Steps:

1. Open a new PowerShell window.
2. Run `docker version` and confirm both Client and Server sections appear.
3. Run `docker info` and identify: number of containers, number of images, storage driver, kernel version, total memory.
4. Run `docker run --rm hello-world`.
5. Run `docker run --rm -it ubuntu bash`. This drops you into an interactive shell inside an Ubuntu container. Type `cat /etc/os-release` to prove you are inside Ubuntu, then type `exit` to leave.
6. Confirm the ubuntu container was auto-removed by running `docker ps -a` — it should not appear (the `--rm` flag removed it on exit).

Acceptance criteria:

- ☐ `docker version` shows both Client and Server sections.
- ☐ `docker info` runs without error and reports at least 1 image.
- ☐ `hello-world` container printed the "Hello from Docker!" message.
- ☐ You successfully entered an Ubuntu container's shell and confirmed the OS.
- ☐ `docker ps -a` shows the ubuntu container is gone (thanks to `--rm`).

Mini-Project

1.2.2.0.25 🎯 Project Title Docker Environment Health Check Script

1.2.2.0.26 🎯 Objective Build a reusable PowerShell script (`docker-healthcheck.ps1`) that anyone on your team could run on a fresh Windows laptop to determine if their Docker installation is healthy. This mirrors what real DevOps engineers do — no one wants to walk 200 developers through the same 10-step manual check individually.

1.2.2.0.27 📋 Requirements Your script must:

1. **Check Docker is installed** — verify the `docker` command is on the PATH.
2. **Print the Docker client version.**
3. **Check the Docker daemon is reachable** — try `docker info` and detect failure.
4. **Print the Docker server (daemon) version.**
5. **Report WSL2 status** — run `wsl --status` and print result.
6. **Report resources allocated** — CPUs and memory from `docker info`.
7. **Run `hello-world`** with `--rm` and capture whether it printed the success message.
8. **Print a color-coded summary report** at the end: green ✅ for each check that passed, red ❌ for each that failed, with a final overall PASS/FAIL verdict.
9. **Exit code** — the script must exit with code `0` if everything passed and `1` if anything failed (so it can be used in CI).

1.2.2.0.28 🪜 Step-by-Step Guidance

1. Create `docker-mastery\milestone-2\docker-healthcheck.ps1`.
2. Start each check as its own PowerShell function returning `$true` or `$false` and pushing a message to a `$results` array.
3. Use `Get-Command docker -ErrorAction SilentlyContinue` to test presence.
4. Wrap each Docker call in `try { ... } catch { ... }` so a failure doesn't crash the whole script.
5. Capture command output with `2>&1` to see both stdout and stderr.
6. For color-coded output, use `Write-Host -ForegroundColor Green` and `Write-Host -ForegroundColor Red`.
7. At the end, count failures and `exit ($failures -gt 0 ? 1 : 0)`.
8. Test the happy path (Docker running). Then quit Docker Desktop and rerun — the script should detect the daemon failure and exit 1.

1.2.2.0.29 □ Complete Mini-Project Solution □ Complete Mini-Project Solution

```powershell
# docker-healthcheck.ps1
# Docker Environment Health Check
# Usage: powershell -ExecutionPolicy Bypass -File .\docker-healthcheck.ps1

$results = @()
$failures = 0

function Add-Result {
    param([string]$Name, [bool]$Passed, [string]$Detail)
    $script:results += [PSCustomObject]@{
        Name   = $Name
        Passed = $Passed
        Detail = $Detail
    }
    if (-not $Passed) { $script:failures++ }
}

Write-Host ""
Write-Host "==== Docker Environment Health Check ====" -ForegroundColor Cyan
Write-Host ""

# 1. Is `docker` on the PATH?
$dockerCmd = Get-Command docker -ErrorAction SilentlyContinue
if ($dockerCmd) {
    Add-Result "Docker CLI installed" $true $dockerCmd.Source
} else {
    Add-Result "Docker CLI installed" $false "docker command not found on PATH"
}

# 2. Docker client version
try {
    $clientVer = docker version --format '{{.Client.Version}}' 2>&1
    Add-Result "Docker client version" $true $clientVer
} catch {
    Add-Result "Docker client version" $false $_.Exception.Message
}

# 3. Docker daemon reachable?
try {
    $info = docker info --format json 2>&1 | ConvertFrom-Json
    Add-Result "Docker daemon reachable" $true "OK"
} catch {
    Add-Result "Docker daemon reachable" $false "Cannot connect to daemon (is Docker Desktop
    $info = $null
}

# 4. Docker server version
if ($info) {
    Add-Result "Docker server version" $true $info.ServerVersion
} else {
    Add-Result "Docker server version" $false "unavailable"
}

# 5. WSL2 status
try {
    $wsl = (wsl --status 2>&1) -join " "
    if ($wsl -match "Default Version:\s*2") {
        Add-Result "WSL2 status" $true "default version 2"
    } else {
        Add-Result "WSL2 status" $false $wsl
    }
```

```powershell
} catch {
    Add-Result "WSL2 status" $false $_.Exception.Message
}

# 6. Resources allocated
if ($info) {
    $cpus  = $info.NCPU
    $memGB = [math]::Round($info.MemTotal / 1GB, 2)
    Add-Result "Allocated resources" $true "CPUs=$cpus  Memory=${memGB}GiB"
} else {
    Add-Result "Allocated resources" $false "unavailable"
}

# 7. Run hello-world
try {
    $hello = docker run --rm hello-world 2>&1
    if ($hello -match "Hello from Docker!") {
        Add-Result "hello-world container" $true "runs successfully"
    } else {
        Add-Result "hello-world container" $false ($hello -join " ")
    }
} catch {
    Add-Result "hello-world container" $false $_.Exception.Message
}

# 8. Print report
Write-Host ""
Write-Host "==== Summary ====" -ForegroundColor Cyan
foreach ($r in $results) {
    if ($r.Passed) {
        Write-Host ("[PASS] {0,-30} : {1}" -f $r.Name, $r.Detail) -ForegroundColor Green
    } else {
        Write-Host ("[FAIL] {0,-30} : {1}" -f $r.Name, $r.Detail) -ForegroundColor Red
    }
}
Write-Host ""

# 9. Exit code
if ($failures -eq 0) {
    Write-Host "OVERALL: PASS  Docker environment is healthy." -ForegroundColor Green
    exit 0
} else {
    Write-Host "OVERALL: FAIL  $failures check(s) failed." -ForegroundColor Red
    exit 1
}
```

Sample run — healthy environment:

```
powershell -ExecutionPolicy Bypass -File .\docker-healthcheck.ps1

==== Docker Environment Health Check ====

==== Summary ====
[PASS] Docker CLI installed       : C:\Program Files\Docker\Docker\resources\bin\docker.exe
[PASS] Docker client version      : 27.3.1
[PASS] Docker daemon reachable    : OK
[PASS] Docker server version      : 27.3.1
[PASS] WSL2 status                : default version 2
[PASS] Allocated resources        : CPUs=8  Memory=7.66GiB
[PASS] hello-world container      : runs successfully

OVERALL: PASS  Docker environment is healthy.
```

Sample run — Docker Desktop quit:

```
==== Summary ====
[PASS] Docker CLI installed      : C:\Program Files\Docker\Docker\resources\bin\docker.exe
[PASS] Docker client version         : 27.3.1
[FAIL] Docker daemon reachable     : Cannot connect to daemon (is Docker Desktop running?)
[FAIL] Docker server version         : unavailable
[PASS] WSL2 status                   : default version 2
[FAIL] Allocated resources           : unavailable
[FAIL] hello-world container         : Cannot connect to the Docker daemon

OVERALL: FAIL  4 check(s) failed.
```

1.2.2.0.30 ✅ Verification Checklist

- ☐ Script file `docker-healthcheck.ps1` exists at `docker-mastery\milestone-2\`.
- ☐ Script runs to completion without a PowerShell parse error.
- ☐ Each of the 7 checks appears in the summary as either [PASS] or [FAIL].
- ☐ "Docker CLI installed" check passes on your machine.
- ☐ "Docker client version" prints a version like `27.x.x`.
- ☐ "Docker daemon reachable" passes when Docker Desktop is running.
- ☐ "Docker server version" prints the daemon version.
- ☐ "WSL2 status" confirms default version 2.
- ☐ "Allocated resources" prints CPU count and memory in GiB.
- ☐ "hello-world container" prints "runs successfully".
- ☐ Final line says `OVERALL: PASS  Docker environment is healthy.` in green.
- ☐ Exit code is `0` when healthy — check with `echo $LASTEXITCODE`.
- ☐ Stopping Docker Desktop and rerunning produces `OVERALL: FAIL` and exit code 1.

1.2.2.0.31 🚀 Bonus Challenges

1. **Add a disk-space check.** Query `docker system df --format json` and warn if the reclaimable space is over 10 GB (suggest `docker system prune`).
2. **Add a network check.** Try `docker run --rm alpine ping -c 1 8.8.8.8` and confirm containers can reach the internet.
3. **Convert the script to output HTML or JSON.** Add a `-Format` parameter that accepts `Text`, `JSON`, or `HTML` and writes the report accordingly — useful for feeding into an internal dashboard.

Scenario (Real-World Use Case)

You are the new lead engineer on a team of 25 developers scattered across three continents. Every Monday, roughly two developers report to Slack that "Docker broke on my machine over the weekend" — sometimes because Windows Update replaced the WSL2 kernel, sometimes because a corporate anti-virus quarantined `dockerd`, sometimes because they filled the disk. Each incident costs you 15–45 minutes of hand-holding, and you have burned an entire day per month on Docker troubleshooting.

You take the health-check script from this milestone, expand it with three more checks (disk, network, corporate registry connectivity), publish it to the team's internal wiki, and add a rule to your Slack triage bot: "If you say 'Docker is broken', the bot replies with a link to the script and asks you to paste the output." Within two weeks, the average Docker-related interruption drops from 30 minutes to 3 minutes because 90% of issues are self-diagnosed by the script's `[FAIL]` lines.

That is what installation and verification skills look like at production scale: not typing `docker version` once, but automating the health checks so an entire team can self-serve.

Checkpoint Quiz

Question 1. Why does Docker Desktop for Windows install a WSL2 distribution called `docker-desktop`?

Click to reveal answer

Because Linux containers require a Linux kernel, and Windows does not natively provide one. `docker-desktop` is a minimal Linux VM (delivered as a WSL2 distro) that provides the Linux kernel and runs the Docker daemon (`dockerd`). Your Windows-side `docker` CLI talks to that daemon over a socket, giving you the illusion of Linux containers running on Windows.

Question 2. In `docker version` output, what does the `Context: desktop-linux` line under the Client section mean?

Click to reveal answer

A Docker **context** is a named endpoint the CLI talks to. `desktop-linux` is the default context created by Docker Desktop and points to the Linux daemon inside WSL2. You can list contexts with `docker context ls` and switch with `docker context use <name>` — useful when managing multiple remote Docker hosts.

Question 3. True or false: Running `docker run hello-world` after a fresh install must download an image before running the container.

Click to reveal answer

True. Because your local image cache is empty on a fresh install, Docker prints `Unable to find image 'hello-world:latest' locally` and pulls it from Docker Hub. On a second run, the image is cached and only the container is created — much faster and no download output.

Question 4. Your colleague reports that after a Windows Update, `docker version` shows only the Client block and errors out on the Server. What is the most likely fix?

Click to reveal answer

The daemon inside WSL2 is not reachable. Most likely causes: (a) Docker Desktop is not running — launch it from the Start menu; (b) the WSL2 kernel was updated and needs the WSL runtime restarted — run `wsl --shutdown` in PowerShell then relaunch Docker Desktop; (c) in rare cases, run `wsl --update` to install the latest WSL kernel package.

Question 5. You want scripts to work whether the user has Docker Desktop or a remote Docker daemon. Which single field in `docker info --format json` tells you what OS the daemon is running on?

Click to reveal answer

The **`OSType`** field. It will be `linux` for Linux daemons (including Docker Desktop's WSL2 daemon) and `windows` for a Windows-container daemon. You would use it like `docker info --format '{{.OSType}}'` and branch your script accordingly.

⬆ Back to Table of Contents

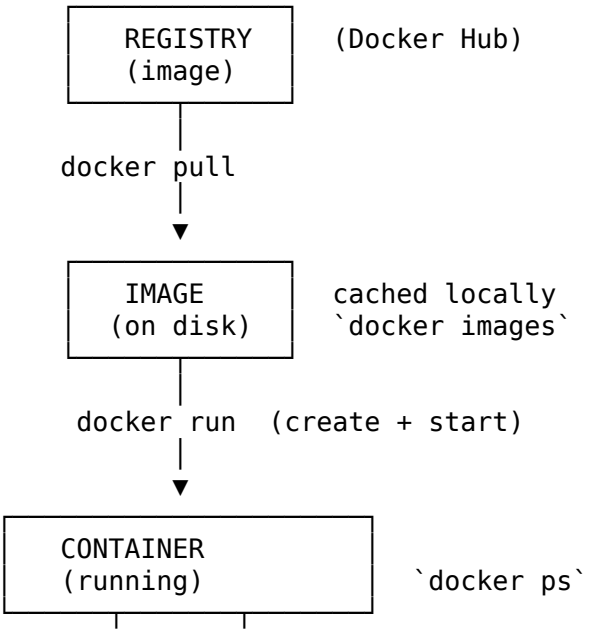
1.2.3 Milestone 3: Docker CLI Basics

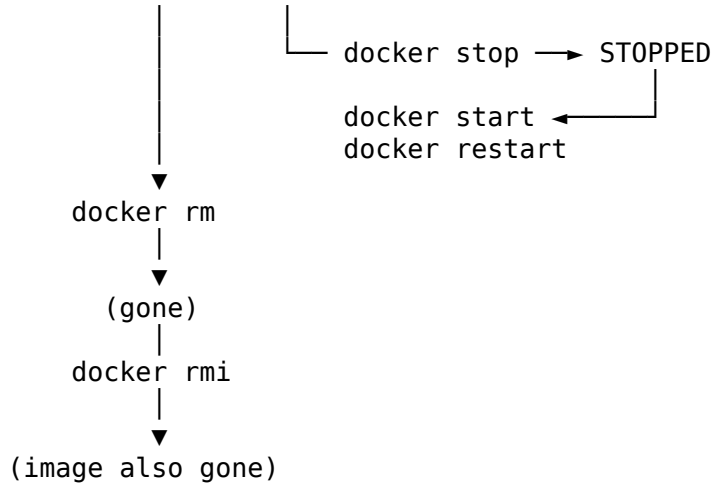
Theory

Now that Docker is installed and healthy, you will learn the seven core CLI commands that make up 90% of daily Docker use: `run`, `ps`, `stop`, `start`, `restart`, `rm`, `pull`, `images`, and `rmi`. These commands cover the entire lifecycle of a container — from downloading its image to creating, inspecting, stopping, restarting, and destroying it — and every advanced topic in later milestones builds on this foundation.

1.2.3.0.1 The Container Lifecycle in One Diagram

<!-- CODE -->





1.2.3.0.2 The Two Modes: Interactive vs Detached Every container runs a single "main process" (its `CMD` or `ENTRYPOINT`, defined in the image). What that process does — and how you interact with it — depends on how you launch it:

- **Interactive mode (`-i -t`, usually combined as `-it`)** attaches your terminal's keyboard and screen to the container's stdin/stdout/stderr, and gives it a pseudo-TTY (so things like colored prompts and Ctrl+C work). Use this to explore a shell, run an interactive REPL (Python, Node, mysql), or step through a script.
  - `-i` (`--interactive`) — keep stdin open, so keyboard input flows into the container.
  - `-t` (`--tty`) — allocate a pseudo-terminal, so shell prompts render properly.
  - You almost always want both together for shells. Using only `-i` works for piping input; using only `-t` is rare.
- **Detached mode (`-d`)** runs the container in the background and returns you to your prompt immediately. The container keeps running until its main process exits. Use this for long-running services like web servers and databases.

You can attach to a detached container later with `docker attach` or `docker exec -it <name> bash`, and you can watch its logs with `docker logs -f`.

1.2.3.0.3 Names and Identifiers Every container has:

- **A container ID** — a 64-character SHA256 hex string. Docker usually shows a shortened 12-character version.
- **A name** — either auto-generated (like `boring_einstein` — a random adjective + scientist) or one you supply with `--name`.

You can refer to a container by full ID, short ID, or name in any command that takes `<container>`. Names are strongly recommended in real work because they make `docker ps`, `docker logs`, and scripts self-documenting.

1.2.3.0.4 Port Mapping By default, container processes are only reachable from *inside* the container. To make a container's port available to the Windows host (and thus a browser), you use `-p HOST_PORT:CONTAINER_PORT`.

- `-p 8080:80` — forward host port 8080 to container port 80. Anyone hitting `http://localhost:8080` reaches the container's port 80.
- `-p 127.0.0.1:8080:80` — bind only to the loopback address on the host, so only your machine can reach it (not other machines on your LAN).
- `-p 8080:80 -p 8443:443` — multiple mappings.
- `-p 80` — publish to a random high port on the host (see the assigned port with `docker port <name>` or `docker ps`).
- `-P` (capital) — publish *all* ports the image `EXPOSE`s to random host ports.

1.2.3.0.5 Environment Variables Applications often read configuration from environment variables (database URLs, API keys, log levels). You pass these in with:

- `-e KEY=value` — a single variable.
- `-e KEY` — pass through a variable from your shell to the container.

- `--env-file .env` — read KEY=value lines from a file. Cleaner for many variables and easier to keep secrets out of shell history.

1.2.3.0.6 The `--rm` Flag Normally, a stopped container hangs around on disk so you can restart it, inspect its logs, or copy files out. During experimentation this quickly clutters your system with dozens of dead containers. Add `--rm` to `docker run` to automatically delete the container the moment its main process exits. Use `--rm` for one-off commands (`docker run --rm alpine echo hi`), and *do not* use it for services (you would lose logs when the container exits).

Commands

- `docker run` — create and start a new container from an image.
- `docker ps` — list running containers.
- `docker ps -a` — list all containers, including stopped ones.
- `docker stop` — send SIGTERM to a container's main process (then SIGKILL after a grace period).
- `docker start` — start a stopped container.
- `docker restart` — stop then start a container.
- `docker rm` — delete a stopped (or forced-stopped) container.
- `docker pull` — download an image from a registry into the local cache.
- `docker images` (alias: `docker image ls`) — list locally cached images.
- `docker rmi` (alias: `docker image rm`) — delete a locally cached image.
- `docker port` — show port mappings of a container.
- `docker logs` — print a container's captured stdout/stderr.
- `docker exec` — run a new command inside a running container.

Command Options/Flags

1.2.3.0.7 `docker run` flags (most-used subset — full list has 100+ flags)

Flag	Short Form	Description	Example
`--name`	—	Assign a name to the container	`docker run --name web nginx`
`--detach`	`-d`	Run in the background	`docker run -d nginx`
`--interactive`	`-i`	Keep stdin open	`docker run -i alpine`
`--tty`	`-t`	Allocate a pseudo-TTY	`docker run -it alpine sh`
`--rm`	—	Delete container on exit	`docker run --rm alpine echo hi`
`--publish`	`-p`	Map host port to container port	`docker run -p 8080:80 nginx`
`--publish-all`	`-P`	Publish all EXPOSEd ports to random host ports	`docker run -P nginx`
`--env`	`-e`	Set environment variable	`docker run -e LOG_LEVEL=debug nginx`
`--env-file`	—	Load env vars from file	`docker run --env-file .env nginx`
`--volume`	`-v`	Mount a volume or host path	`docker run -v mydata:/data nginx`
`--mount`	—	Attach a volume/bind (verbose syntax)	`docker run --mount type=volume,src=mydata,dst= nginx`
`--workdir`	`-w`	Set working directory inside container	`docker run -w /app node`
`--user`	`-u`	Run as UID/username	`docker run -u 1000 alpine`
`--network`	—	Attach to a specific network	`docker run --network host nginx`
`--hostname`	`-h`	Set container hostname	`docker run -h web01 nginx`

Flag	Short Form	Description	Example
`--restart`	—	Restart policy (no, on-failure, always, unless-stopped)	`docker run --restart=always nginx`
`--memory`	`-m`	Memory limit	`docker run -m 512m nginx`
`--cpus`	—	CPU limit (fractional cores)	`docker run --cpus=1.5 nginx`
`--label`	`-l`	Set metadata label	`docker run -l env=dev nginx`
`--entrypoint`	—	Override image ENTRYPOINT	`docker run --entrypoint sh alpine`
`--pull`	—	Always/missing/never pull before run	`docker run --pull=always nginx`
`--init`	—	Run tiny init as PID 1 (zombie reaping)	`docker run --init node`
`--read-only`	—	Mount root filesystem as read-only	`docker run --read-only nginx`
`--tmpfs`	—	Mount a tmpfs at a path	`docker run --tmpfs /tmp nginx`
`--dns`	—	Custom DNS server	`docker run --dns 8.8.8.8 alpine`
`--add-host`	—	Add host:ip to `/etc/hosts`	`docker run --add-host db:10.0.0.1 nginx`
`--platform`	—	Target platform for image (linux/amd64, linux/arm64)	`docker run --platform=linux/arm64 alpine`

1.2.3.0.8 `docker ps` flags

Flag	Short Form	Description	Example
`--all`	`-a`	Show all containers (including stopped)	`docker ps -a`
`--quiet`	`-q`	Only display container IDs	`docker ps -q`
`--filter`	`-f`	Filter output (see below for values)	`docker ps -f status=running`
`--format`	—	Go template for formatting	`docker ps --format '{{.Names}}'`
`--last`	`-n`	Show n last created containers	`docker ps -n 3`
`--latest`	`-l`	Show the latest created container	`docker ps -l`
`--no-trunc`	—	Don’t truncate output (show full IDs)	`docker ps --no-trunc`
`--size`	`-s`	Show container writable-layer size	`docker ps -s`

`docker ps` filter values include: `status=` (running/exited/paused/created/restarting), `name=`, `id=`, `label=key=value`, `ancestor=image`, `network=`, `volume=`, `publish=port`, `expose=port`, `before=name`, `since=name`.

1.2.3.0.9 `docker stop` flags

Flag	Short Form	Description	Example
`--time`	`-t`	Seconds to wait before killing (default 10)	`docker stop -t 30 web`
`--signal`	`-s`	Signal to send (default SIGTERM)	`docker stop -s SIGINT web`

1.2.3.0.10 `docker start` flags

Flag	Short Form	Description	Example
`--attach`	`-a`	Attach stdout/stderr	`docker start -a web`
`--interactive`	`-i`	Attach stdin	`docker start -i web`
`--detach-keys`	—	Override sequence to detach	`docker start --detach-keys="ctrl-p,ctrl-q" web`

1.2.3.0.11 `docker restart` flags

Flag	Short Form	Description	Example
`--time`	`-t`	Seconds to wait before killing before restart	`docker restart -t 5 web`
`--signal`	`-s`	Signal to send	`docker restart -s SIGHUP web`

1.2.3.0.12 `docker rm` flags

Flag	Short Form	Description	Example
`--force`	`-f`	Force-remove a running container (sends SIGKILL)	`docker rm -f web`
`--volumes`	`-v`	Also remove anonymous volumes attached to it	`docker rm -v web`
`--link`	`-l`	Remove specified link (legacy)	`docker rm -l alias`

1.2.3.0.13 `docker pull` flags

Flag	Short Form	Description	Example
`--all-tags`	`-a`	Pull every tag of the repository	`docker pull -a nginx`
`--disable-content-trust`	—	Skip image signature verification	`docker pull --disable-content-trust nginx`
`--platform`	—	Set platform if image is multi-arch	`docker pull --platform=linux/arm64 nginx`
`--quiet`	`-q`	Suppress verbose progress output	`docker pull -q nginx`

1.2.3.0.14 `docker images` / `docker image ls` flags

Flag	Short Form	Description	Example
`--all`	`-a`	Show all images (including intermediate)	`docker images -a`
`--digests`	—	Show image digests	`docker images --digests`
`--filter`	`-f`	Filter output	`docker images -f dangling=true`
`--format`	—	Go template	`docker images --format '{{.Repos-itory}}:{{.Tag}}'`
`--no-trunc`	—	Don’t truncate output	`docker images --no-trunc`
`--quiet`	`-q`	Only show image IDs	`docker images -q`

`docker images` filter values include: `dangling=true|false`, `label=`, `before=image`, `since=image`, `reference=pattern`.

1.2.3.0.15 `docker rmi` / `docker image rm` flags

Flag	Short Form	Description	Example
`--force`	`-f`	Force-remove even if in use by stopped containers	`docker rmi -f nginx`
`--no-prune`	—	Don’t delete untagged parent images	`docker rmi --no-prune nginx`

Examples

1.2.3.0.16 Example 1 — Run a container in detached mode and inspect it

```
docker run -d --name web -p 8080:80 nginx:alpine
```

```
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
c6b39d5da001: Pull complete
27f5a1b7e34c: Pull complete
...
Digest: sha256:2140dad235c1...
Status: Downloaded newer image for nginx:alpine
d6f2a4e3b1c9a0f8e2d7c4b1a09e8f7d6c5b4a3d2e1f0987654321abcdef1234
```

The long hex string is the new container’s full ID. Now list it:

```
docker ps
```

```
CONTAINER ID  IMAGE       COMMAND              CREATED      STATUS       PORTS             NAMES
d6f2a4e3b1c9  nginx:alpine  "/docker-entrypoint.…"  3 seconds ago  Up 2 seconds  0.0.0.0:8080-
>80/tcp   web
```

Open `http://localhost:8080` in your browser — you see the Nginx welcome page.

1.2.3.0.17 Example 2 — Interactive shell in Alpine

```
docker run -it --rm alpine sh
```

```
/ # cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.20.3
PRETTY_NAME="Alpine Linux v3.20"
HOME_URL="https://alpinelinux.org/"
```

```
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/issues"
/ # exit
```

Because of `--rm`, the container is gone the moment you type `exit`.

1.2.3.0.18 Example 3 — Environment variables

```
docker run --rm -e GREETING=hello -e NAME=world alpine sh -c 'echo "$GREETING $NAME"'
```

```
hello world
```

1.2.3.0.19 Example 4 — Stop, start, restart

```
docker stop web
docker ps -a
docker start web
docker restart web
```

```
web
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS                      PORTS     NAMES
d6f2a4e3b1c9   nginx:alpine   "/docker-entrypoint.…"   3 minutes ago    Exited (0) 5 seconds ago              web
web
web
```

1.2.3.0.20 Example 5 — Remove containers and images

```
docker rm -f web
docker rmi nginx:alpine
```

```
web
Untagged: nginx:alpine
Untagged: nginx@sha256:2140dad235c1...
Deleted: sha256:d1a364dc548d...
Deleted: sha256:78b5f3f45f39...
...
```

Command Exercises

1.2.3.0.21 Exercise 3.1 — `docker run` basic Run an Alpine container that prints `hello` and immediately exits.

▯ Solution

```
docker run alpine echo hello
```

```
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
Digest: sha256:....
Status: Downloaded newer image for alpine:latest
hello
```

The image is downloaded on first use, the container runs `echo hello`, prints the word, then exits with code 0. The stopped container remains on disk until you `docker rm` it.

1.2.3.0.22 Exercise 3.2 — `docker run` with detached + named + port Run Nginx detached, name the container `mysite`, and expose port 80 as host port 9090.

▯ Solution

```
docker run -d --name mysite -p 9090:80 nginx:alpine
```

```
b4d2e6a7f1c30987654321fedcba9876543210abcdef1234567890abcdef1234
```

Verify with `docker ps` and open http://localhost:9090 — the Nginx welcome page appears.

1.2.3.0.23 Exercise 3.3 — `docker run` combining flags and troubleshooting Try to start a second container also on host port 9090 named `mysite2`. Observe the error, then successfully start it on a different host port.

 Solution

```
docker run -d --name mysite2 -p 9090:80 nginx:alpine
```

```
docker: Error response from daemon: driver failed programming external
connectivity on endpoint mysite2: Bind for 0.0.0.0:9090 failed: port is
already allocated.
```

Diagnosis: only one process can bind host port 9090. Fix by using a different host port; the container port can stay 80:

```
docker run -d --name mysite2 -p 9091:80 nginx:alpine
docker ps
```

```
CONTAINER ID   IMAGE          PORTS                   NAMES
b4d2e6a7f1c3   nginx:alpine   0.0.0.0:9090->80/tcp   mysite
c9e3f4d8a2b1   nginx:alpine   0.0.0.0:9091->80/tcp   mysite2
```

Both containers now run; hitting :9090 and :9091 shows the same page from two independent Nginx containers.

1.2.3.0.24 Exercise 3.4 — `docker ps` basic List only running containers.

 Solution

```
docker ps
```

```
CONTAINER ID   IMAGE         COMMAND                  STATUS        PORTS                NAMES
b4d2e6a7f1c3   nginx:alpine   "/docker-entrypoint.…"   Up 5 minutes   0.0.0.0:9090-
>80/tcp   mysite
c9e3f4d8a2b1   nginx:alpine   "/docker-entrypoint.…"   Up 2 minutes   0.0.0.0:9091-
>80/tcp   mysite2
```

1.2.3.0.25 Exercise 3.5 — `docker ps` with filters and format Show ONLY the names and ports of containers whose image is `nginx:alpine`.

 Solution

```
docker ps --filter ancestor=nginx:alpine --format 'table {{.Names}}\t{{.Ports}}'
```

```
NAMES     PORTS
mysite    0.0.0.0:9090->80/tcp
mysite2   0.0.0.0:9091->80/tcp
```

The `ancestor` filter matches containers created from a specific image.

1.2.3.0.26 Exercise 3.6 — `docker ps -a` and quiet mode Get only the IDs of all containers (running AND stopped) so you can pass them to another command.

 Solution

```
docker ps -a -q
```

```
b4d2e6a7f1c3
c9e3f4d8a2b1
5e6f7a8b9c01
```

This is the idiomatic way to script bulk operations. For example:

```
docker rm -f $(docker ps -a -q)
```

removes every container on the host. Use with caution.

1.2.3.0.27 Exercise 3.7 — `docker stop` default and custom grace period

Stop `mysite` with the default 10-second SIGTERM grace period.

 Solution

```
docker stop mysite
```

```
mysite
```

Docker sends SIGTERM to the main process. Nginx's master handles it and exits gracefully within a fraction of a second, so `stop` returns almost instantly. If a process ignores SIGTERM for 10 seconds, Docker sends SIGKILL.

1.2.3.0.28 Exercise 3.8 — `docker stop` with custom timeout

Stop `mysite2` with only a 2-second grace period.

 Solution

```
docker stop -t 2 mysite2
```

```
mysite2
```

Useful when a container is misbehaving and you don't want to wait 10 seconds.

1.2.3.0.29 Exercise 3.9 — `docker stop` on already-stopped container

Try to stop a container that's already stopped. Observe.

 Solution

```
docker stop mysite
docker stop mysite
```

```
mysite
mysite
```

Stopping a stopped container is a no-op — it succeeds silently and prints the name. This is idempotent behavior, safe for scripts.

1.2.3.0.30 Exercise 3.10 — `docker start` basic

Start `mysite` again after stopping it.

 Solution

```
docker start mysite
docker ps
```

```
mysite
CONTAINER ID   IMAGE          STATUS          NAMES
b4d2e6a7f1c3   nginx:alpine   Up 2 seconds    mysite
```

`docker start` restarts an existing stopped container. All its configuration (name, ports, env vars, volumes) is preserved.

1.2.3.0.31 Exercise 3.11 — `docker start` attached

Create and immediately stop a small container, then use `docker start -a` to run it and see its output.

 Solution

```
docker create --name counter alpine sh -c 'for i in 1 2 3; do echo tick $i; sleep 1; done'
docker start -a counter
```

```
counter
tick 1
tick 2
tick 3
```

`-a` attaches your terminal so you see the container's output as it prints. Without `-a`, `docker start` returns immediately and the output goes only to the logs (accessible via `docker logs counter`).

1.2.3.0.32 Exercise 3.12 — `docker rm` basic

Remove the stopped `counter` container.

 Solution

```
docker rm counter
```

```
counter
```

Only stopped containers can be removed by default. Verify with `docker ps -a` — `counter` no longer appears.

1.2.3.0.33 Exercise 3.13 — `docker rm -f` (force)

Try to remove a RUNNING container without `-f`, observe the error, then force-remove it.

 Solution

```
docker rm mysite
```

```
Error response from daemon: cannot remove container "/mysite": container is
running: stop the container before removing or force remove
```

```
docker rm -f mysite
```

```
mysite
```

`-f` sends SIGKILL to the container then removes it. Skip the graceful shutdown — useful when you don't care about clean exit.

1.2.3.0.34 Exercise 3.14 — `docker rm` bulk

Create three throwaway alpine containers that exit immediately, then remove them all in one command.

 Solution

```
docker run --name t1 alpine echo done
docker run --name t2 alpine echo done
docker run --name t3 alpine echo done
docker rm t1 t2 t3
```

```
done
done
done
t1
t2
t3
```

`docker rm` accepts multiple names/IDs in one call. In scripts you often combine with `docker ps -aq`:

```
docker rm $(docker ps -aq --filter status=exited)
```

removes every stopped container.

1.2.3.0.35 Exercise 3.15 — `docker pull` basic

Pull the `busybox` image explicitly (before running any container).

 Solution

```
docker pull busybox
```

```
Using default tag: latest
latest: Pulling from library/busybox
c3f5a5c92cb0: Pull complete
Digest: sha256:34b191d63fbc93e25e275bccccdaf3b3d4f4d2f95ab0d0d5c1b96d5d78e26ea1
Status: Downloaded newer image for busybox:latest
docker.io/library/busybox:latest
```

Because no tag was given, Docker defaulted to `:latest`.

1.2.3.0.36 Exercise 3.16 — `docker pull` specific tag

Pull `python:3.12-slim` explicitly.

Solution

```
docker pull python:3.12-slim
```

```
3.12-slim: Pulling from library/python
c1ec31eb5944: Already exists
5a1c1e8dfd7b: Pull complete
...
Digest: sha256:abc123...
Status: Downloaded newer image for python:3.12-slim
docker.io/library/python:3.12-slim
```

`Already exists` on the first layer means Docker recognized that layer from a previous pull (perhaps `debian` or `nginx:alpine`) — layers are shared and only downloaded once.

1.2.3.0.37 Exercise 3.17 — `docker pull --platform`

Pull the ARM64 variant of alpine on your x86 laptop (useful for testing multi-arch builds).

Solution

```
docker pull --platform=linux/arm64 alpine:3.20
```

```
3.20: Pulling from library/alpine
71fc0a10c1a1: Pull complete
Digest: sha256:....
Status: Downloaded newer image for alpine:3.20
docker.io/library/alpine:3.20
```

You now have the arm64 variant cached. Trying to run it directly on Windows x86_64 would use QEMU emulation via Docker Desktop's rosetta integration — much slower, but works.

1.2.3.0.38 Exercise 3.18 — `docker images` basic

List all images on your machine.

Solution

```
docker images
```

```
REPOSITORY     TAG          IMAGE ID       CREATED        SIZE
nginx          alpine       d1a364dc548d   3 days ago     20.5MB
python         3.12-slim    5a3c1e8f9a10   1 week ago     129MB
alpine         latest       f8c20f8bbcb6   2 weeks ago    7.7MB
alpine         3.20         71fc0a10c1a1   2 weeks ago    7.7MB
busybox        latest       034ac2d9bcbb   4 weeks ago    4.5MB
hello-world    latest       d2c94e258dcb   6 months ago   13.3kB
```

1.2.3.0.39 Exercise 3.19 — `docker images` filtered and formatted

Show ONLY the repository:tag and size of images larger than nothing, sorted by size descending — using `--format`.

Solution

```
docker images --format 'table {{.Repository}}:{{.Tag}}\t{{.Size}}'
```

```
REPOSITORY:TAG        SIZE
nginx:alpine          20.5MB
python:3.12-slim      129MB
alpine:latest         7.7MB
alpine:3.20           7.7MB
busybox:latest        4.5MB
hello-world:latest    13.3kB
```

For true sorting you would need to pipe to PowerShell `Sort-Object`, but this format is much cleaner for scripts.

1.2.3.0.40 Exercise 3.20 — `docker images -q` for scripting

Get just the image IDs so you could feed them to another command.

Solution

```
docker images -q
```

```
d1a364dc548d
5a3c1e8f9a10
f8c20f8bbcb6
71fc0a10c1a1
034ac2d9bcbb
d2c94e258dcb
```

Combine with `docker rmi $(docker images -q)` for a nuclear cleanup — but be careful, that removes everything.

1.2.3.0.41 Exercise 3.21 — `docker rmi` basic

Remove the `busybox:latest` image.

▸ Solution

```
docker rmi busybox:latest
```

```
Untagged: busybox:latest
Untagged: busybox@sha256:34b191d63fbc93e25e275bccccdaf3b3d4f4d2f95ab0d0d5c1b96d5d78e26ea1
Deleted: sha256:034ac2d9bcbb...
Deleted: sha256:1c1c1cbcbb...
```

`Untagged` means the tag `busybox:latest` and its digest reference were removed. `Deleted` means the actual layer blobs were reclaimed from disk because no other image referenced them.

1.2.3.0.42 Exercise 3.22 — `docker rmi` in-use error

Try to remove an image that a stopped container was created from. Observe the error, then force it.

▸ Solution

```
docker run --name web nginx:alpine
docker stop web
docker rmi nginx:alpine
```

```
Error response from daemon: conflict: unable to remove repository reference
"nginx:alpine" (must force) - container 8f1e2b3c4d5e is using its referenced
image d1a364dc548d
```

Fix: force-remove or first delete the container.

```
docker rmi -f nginx:alpine
```

```
Untagged: nginx:alpine
Deleted: sha256:d1a364dc548d...
```

Note: `-f` removes only the image *tag*. The container `web` (stopped) still exists and its writable layer still consumes disk. `docker ps -a` will show the container as `<none>:<none>`. Clean up with `docker rm web`.

1.2.3.0.43 Exercise 3.23 — `docker rmi` dangling images

Find and remove all “dangling” images (images with no tag, usually left by rebuilds).

▸ Solution

```
docker images -f dangling=true -q
docker rmi $(docker images -f dangling=true -q)
```

```
8f7c6b5a4d3e
2e1f0d9c8b7a

Deleted: sha256:8f7c6b5a4d3e...
Deleted: sha256:2e1f0d9c8b7a...
```

Or, more idiomatically:

```
docker image prune -f
```

```
Total reclaimed space: 156.2MB
```

Both approaches accomplish the same thing.

Hands-On Assignment

Task: Run Nginx in a container, browse to it, view its logs, stop it, and clean up.

Steps and expected outputs:

1. Pull nginx image:

   ```
   docker pull nginx:alpine
   ```

   ```
   alpine: Pulling from library/nginx
   ... Pull complete ...
   Status: Downloaded newer image for nginx:alpine
   ```

2. Run detached with port mapping and a name:

   ```
   docker run -d --name my-nginx -p 8080:80 nginx:alpine
   ```

   ```
   a1b2c3d4e5f6789...
   ```

3. Verify running:

   ```
   docker ps
   ```

   ```
   CONTAINER ID   IMAGE          PORTS                   NAMES
   a1b2c3d4e5f6   nginx:alpine   0.0.0.0:8080->80/tcp   my-nginx
   ```

4. Open **http://localhost:8080** in browser → see “Welcome to nginx!” page.

5. View recent logs (Nginx logs the incoming request):

   ```
   docker logs my-nginx
   ```

   ```
   /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configur
   ...
   2025/xx/xx 12:34:56 [notice] 1#1: start worker processes
   172.17.0.1  -  -  [xx/xxx/2025:12:35:01  +0000]  "GET  /  HTTP/1.1"  200  615  "-
   " "Mozilla/5.0 ..."
   ```

6. Stop the container:

   ```
   docker stop my-nginx
   ```

7. Remove it:

   ```
   docker rm my-nginx
   ```

8. Confirm gone:

   ```
   docker ps -a
   ```

Acceptance criteria:

- ☐ `nginx:alpine` image is pulled.
- ☐ Container starts detached with name `my-nginx` and port mapping `8080:80`.
- ☐ Browser at `http://localhost:8080` shows Nginx welcome page.
- ☐ `docker logs my-nginx` shows at least one GET request from your browser.
- ☐ Container stops without force.
- ☐ Container is removed and does not appear in `docker ps -a`.

Mini-Project

1.2.3.0.44 □ Project Title Personal Container Playground Manager

1.2.3.0.45 □ Objective Practice the complete container lifecycle by running, managing, and cleaning up three real-world containers — a web server (Nginx), a second web server (Apache/httpd), and a data store (Redis) — using every CLI command from this milestone. You will document every command and its output, producing a lab notebook that proves you can operate Docker end-to-end.

1.2.3.0.46 ✅ Requirements

1. **Pull three images:** `nginx:alpine`, `httpd:alpine`, `redis:alpine` — using `docker pull`.
2. **Run all three as detached named containers** with these specific mappings:
   - Nginx: name `pg-nginx`, port `8081:80`, env `SITE_NAME=playground`.
   - Apache: name `pg-httpd`, port `8082:80`, env `SERVER_ROLE=api-mock`.
   - Redis: name `pg-redis`, port `6379:6379`.
3. **Verify all three are running** with a single `docker ps` and a filtered `docker ps` that uses `--format`.
4. **Browse** to `http://localhost:8081` (Nginx) and `http://localhost:8082` (Apache) and confirm both welcome pages appear.
5. **Enter Redis interactively** with `docker exec -it pg-redis redis-cli` and run `SET foo bar` then `GET foo`.
6. **Stop only `pg-httpd`**, verify with `docker ps` (should show only 2 running) and `docker ps -a` (should show 3, one exited).
7. **Restart `pg-httpd`** and verify all three are again running.
8. **Remove all three containers** (force where required).
9. **Remove all three images.**
10. **Verify a clean state:** `docker ps -a` shows no playground containers; `docker images` shows none of the three.

Document every command and expected output in a Markdown file called `playground-lab.md`.

1.2.3.0.47 ✅ Step-by-Step Guidance

1. Create `docker-mastery\milestone-3\playground-lab.md` and open it in VS Code.
2. Write a "## Setup" section — run each `docker pull` command and paste its output.
3. Write a "## Run" section — three `docker run -d ...` commands with expected container IDs.
4. Write a "## Verify" section — outputs of `docker ps` and a formatted variant.
5. Write a "## Browser Test" section — screenshots or a note that both welcome pages appeared.
6. Write a "## Redis Test" section — the `docker exec` command, the SET/GET output, and the `exit` step.
7. Write a "## Stop and Restart" section — `docker stop`, verification, `docker start` / `restart`, verification.
8. Write a "## Cleanup" section — `docker rm -f`, `docker rmi`, and the final proof of a clean state.
9. Reflect briefly at the end — three things you learned that you didn't know at the start.

1.2.3.0.48 ✅ Complete Mini-Project Solution ✅ Complete Mini-Project Solution

```
# Personal Container Playground Manager — Lab

## Setup — Pull images

```powershell
docker pull nginx:alpine
docker pull httpd:alpine
docker pull redis:alpine
```

Output (abridged):
```
alpine: Pulling from library/nginx
Status: Downloaded newer image for nginx:alpine

alpine: Pulling from library/httpd
Status: Downloaded newer image for httpd:alpine

alpine: Pulling from library/redis
Status: Downloaded newer image for redis:alpine
```

Run — Three detached, named containers

```powershell
```
```

```
docker run -d --name pg-nginx -p 8081:80 -e SITE_NAME=playground nginx:alpine
docker run -d --name pg-httpd -p 8082:80 -e SERVER_ROLE=api-mock httpd:alpine
docker run -d --name pg-redis -p 6379:6379 redis:alpine
```

Output:
```
a1b2c3d4e5f67890...
b2c3d4e5f6789012...
c3d4e5f678901234...
```

Verify — All three running

```powershell
docker ps
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Ports}}\t{{.Status}}'
```

Output:
```
CONTAINER ID   IMAGE          PORTS                     NAMES
a1b2c3d4e5f6   nginx:alpine   0.0.0.0:8081->80/tcp      pg-nginx
b2c3d4e5f678   httpd:alpine   0.0.0.0:8082->80/tcp      pg-httpd
c3d4e5f67890   redis:alpine   0.0.0.0:6379->6379/tcp    pg-redis

NAMES       IMAGE          PORTS                     STATUS
pg-nginx    nginx:alpine   0.0.0.0:8081->80/tcp      Up 30 seconds
pg-httpd    httpd:alpine   0.0.0.0:8082->80/tcp      Up 25 seconds
pg-redis    redis:alpine   0.0.0.0:6379->6379/tcp    Up 20 seconds
```

Browser Test

- http://localhost:8081 → "Welcome to nginx!"
- http://localhost:8082 → "It works!"

Redis Test — Interactive exec

```powershell
docker exec -it pg-redis redis-cli
```

Inside the Redis shell:
```
127.0.0.1:6379> SET foo bar
OK
127.0.0.1:6379> GET foo
"bar"
127.0.0.1:6379> exit
```

Stop and Restart

```powershell
docker stop pg-httpd
docker ps                # only 2 running
docker ps -a             # 3 exist, 1 exited
docker start pg-httpd
docker restart pg-nginx  # restart just to demonstrate
docker ps                # all 3 running again
```

Output:
```
pg-httpd
(2 rows shown in docker ps)
(3 rows shown in docker ps -a; pg-httpd status = Exited (0))
pg-httpd
pg-nginx
(3 rows shown again in docker ps)
```

Cleanup — Remove containers and images

```powershell
docker rm -f pg-nginx pg-httpd pg-redis
docker rmi nginx:alpine httpd:alpine redis:alpine
docker ps -a
docker images
```

Output:
```
pg-nginx
pg-httpd
pg-redis
Untagged: nginx:alpine
Deleted: sha256:...
Untagged: httpd:alpine
Deleted: sha256:...
Untagged: redis:alpine
Deleted: sha256:...
(docker ps -a: no playground containers)
(docker images: no playground images)
```

Reflections

1. `-p HOST:CONTAINER` uses the *host* port on the left. I had them backward at first and got confusing 404s.
2. `docker exec -it <name> <cmd>` runs a *new* process in a running container, unlike `docker run` which creates a new container.
3. `docker rm -f` is faster than `stop`+`rm` but skips graceful shutdown — fine for playground, dangerous in production for stateful services like Redis (data-loss risk).

1.2.3.0.49 ✅ Verification Checklist

- ☐ `playground-lab.md` exists at `docker-mastery\milestone-3\`.
- ☐ `docker pull` was used for all three images.
- ☐ All three containers were started with `docker run -d`, `--name`, `-p`, and (for two of them) `-e`.
- ☐ `docker ps` showed three running containers with the correct names, ports, and images.
- ☐ `docker ps --format` was used at least once with a Go template.
- ☐ Nginx welcome page opened at `localhost:8081`.
- ☐ Apache welcome page opened at `localhost:8082`.
- ☐ `docker exec -it pg-redis redis-cli` was used and SET/GET succeeded.
- ☐ `docker stop pg-httpd` succeeded and `docker ps -a` showed one exited container.
- ☐ `docker start` (or `restart`) brought `pg-httpd` back up.
- ☐ `docker rm -f` removed all three containers.
- ☐ `docker rmi` removed all three images.
- ☐ Final `docker ps -a` and `docker images` confirmed a clean state.
- ☐ At least three personal reflections were recorded.

1.2.3.0.50 🌟 Bonus Challenges

1. **Add a Postgres container** — pull `postgres:16-alpine`, run with `POSTGRES_PASSWORD=devsecret`, exec into it and run `psql -U postgres -c "SELECT version();"`.
2. **Restart policies** — recreate all three containers with `--restart=unless-stopped`. Quit Docker Desktop and relaunch it; verify the containers come back up automatically.
3. **Resource limits** — recreate the Nginx container with `--memory=64m --cpus=0.25`. Prove the limits are in place with `docker stats pg-nginx` and observe CPU/memory usage against the limits.

Scenario (Real-World Use Case)

You are on-call for a small startup. At 2 AM Slack pings you: "The staging API is down." You SSH into the staging VM, run `docker ps`, and see the `api` container is missing. You run `docker ps -a --filter name=api` and see it exited 12 minutes ago with exit code 137 (SIGKILL — likely OOM-killed). You check `docker logs --tail 50 api` and confirm a `fatal: out of memory` line just before exit.

You bring it back with `docker start api` and it stays up for 4 minutes before dying again — same cause. Rather than wake up the backend team, you restart it with a higher memory limit: `docker rm -f api` then `docker run -d --name api --memory=1g --restart=unless-stopped -p 8080:80 ourcompany/api:staging`. It stays up. In the morning you file a ticket with the log excerpt so the team can find the memory leak, then go back to sleep.

Every command you used in that incident — `ps`, `ps -a --filter`, `logs`, `start`, `rm -f`, run with flags — came from Milestone 3. The rest of Docker mastery is layered on top of this fluency. Practice these commands until they are muscle memory.

Checkpoint Quiz

Question 1. What is the difference between `-i`, `-t`, and `-it`?

Click to reveal answer

`-i` (`--interactive`) keeps stdin open so keyboard input is forwarded to the container. `-t` (`--tty`) allocates a pseudo-terminal so the container thinks it's talking to a real terminal (colors, prompts, Ctrl+C behave). `-it` combines both and is the standard combination for shells. Use `-i` alone for piping input to a container (`echo hi | docker run -i alpine cat`), and `-it` together for anything you'd type into interactively.

Question 2. In `-p 8080:80`, which side is the host and which is the container?

Click to reveal answer

HOST_PORT:CONTAINER_PORT — the LEFT number (`8080`) is the port on your Windows host, the RIGHT number (`80`) is the port inside the container. Your browser talks to `localhost:8080`, Docker forwards that traffic to port 80 inside the container. Getting these backward is one of the most common beginner mistakes.

Question 3. What is the difference between `docker stop` and `docker rm -f`?

Click to reveal answer

`docker stop` sends SIGTERM (a polite "please shut down") to the container's main process, waits up to 10 seconds by default, then sends SIGKILL if it hasn't exited — but the container remains on disk with its logs and configuration intact and can be restarted with `docker start`. `docker rm -f` sends SIGKILL immediately (no graceful shutdown) *and* deletes the container. Use `stop` for services you might restart; use `rm -f` when you're truly done with the container.

Question 4. Why does `docker rmi nginx:alpine` sometimes fail with "conflict"?

Click to reveal answer

Because a container (running OR stopped) was created from that image. Docker tracks the reference — deleting the image would corrupt the container's layer chain. Either `docker rm` the containers first, or use `docker rmi -f` to force-untag the image (leaving the container's copy of the layers intact until the container itself is deleted). Use `docker ps -a --filter ancestor=nginx:alpine` to find offending containers.

Question 5. Predict the output of these two commands run back-to-back:

```
docker run --rm -d --name x alpine sleep 30
docker ps -a
```

Click to reveal answer

The first command launches `alpine sleep 30` detached with the name `x` and `--rm`. It prints the container ID and returns immediately. The container will run in the background for 30 seconds. The second command lists containers, and `x` will appear with STATUS `Up X seconds`. If you wait 30 seconds and rerun `docker ps -a`, the container `x` is *gone* — because `--rm` deleted it automatically when `sleep` exited. Without `--rm`, it would appear as `Exited (0)`.

⬆ Back to Table of Contents

1.2.4 Milestone 4: Docker Images Deep Dive

Theory

You've been *using* images. Now you'll understand what they actually are: stacks of read-only filesystem layers, addressed by cryptographic hashes, distributed through registries, and cached with per-layer granularity. Everything about how images build, ship, and store efficiently comes from this layer model.

1.2.4.0.1 What Is an Image, Precisely? A Docker image is a bundle of two things:

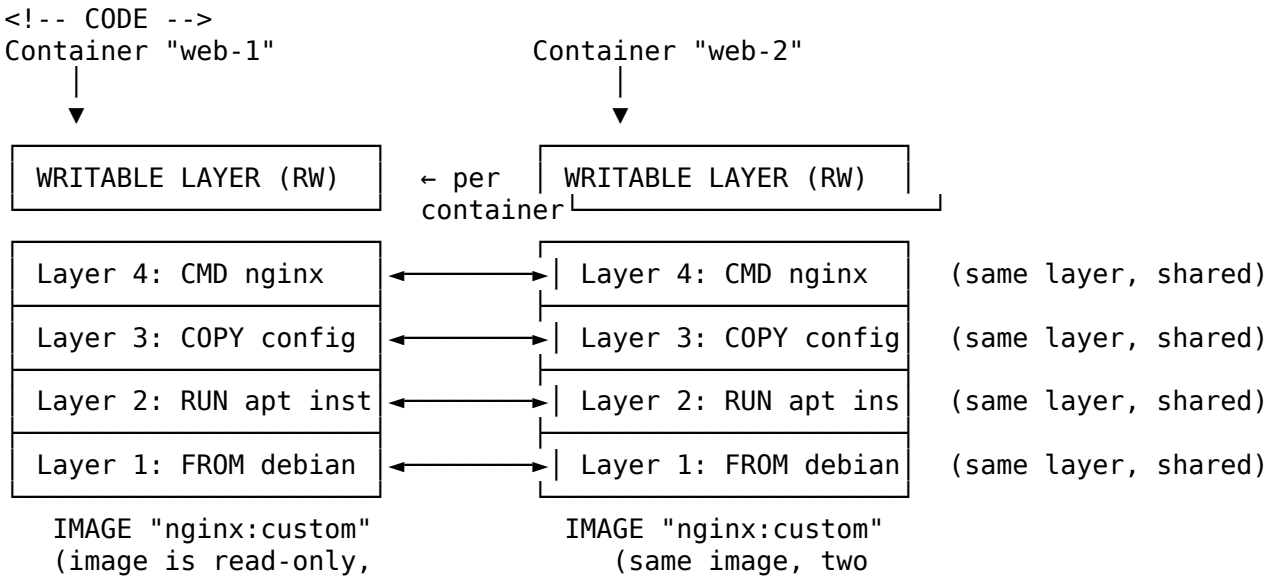
1. **A stack of read-only layers.** Each layer is a tarball of files representing a *change* to the filesystem — files added, modified, or marked as deleted. Layers stack from bottom (base) to top (most recent), and Docker's storage driver (`overlay2` on Linux) merges them into a single unified view at runtime using an **overlay filesystem**.
2. **A manifest (JSON metadata).** This describes the layer stack, the default command to run (`CMD`/`ENTRYPOINT`), exposed ports, environment variables, working directory, labels, and the target CPU architecture and OS.

When you run a container, Docker adds a *writable* layer on top of the read-only stack. Writes go into that writable layer via **copy-on-write** — reading a file traverses layers top-down until found; writing to a file copies it up into the writable layer first. When the container is deleted, the writable layer is discarded — that's why containers are "ephemeral" and why anything you want to survive belongs in a volume (Milestone 7).

1.2.4.0.2 Analogy — Layers Are Like Transparent Overlay Sheets Imagine you're printing a color picture on a transparency machine that only prints one color per pass. You start with a blank sheet (the base), pass 1 adds cyan, pass 2 adds magenta on top, pass 3 adds yellow, pass 4 adds the black outlines. When you hold the finished stack up to the light, you see the merged image — but each sheet by itself is separate and reusable.

Docker images work the same way: each Dockerfile instruction produces one transparent sheet. If two images share the first three sheets (say, both are built from `python:3.12-slim`), Docker keeps only *one* physical copy of those sheets on disk and both images just reference them. That is why pulling a second Python image is much faster than the first — most of the layers are already cached.

1.2.4.0.3 Layered Filesystem — ASCII Diagram



```
a stack of shared layers)          separate containers)
```

1.2.4.0.4 Image Caching — The Rule Nobody Forgets Twice Docker builds and pulls layers using a **content-addressable cache** keyed on the *content hash* of each layer plus the *instruction that produced it*. When you rebuild an image and step 4 changes, steps 1–3 hit the cache and steps 4 onward rebuild. This is why the order of instructions in a Dockerfile matters enormously — you put slow, rarely changing steps early (installing OS packages) and fast, frequently changing steps late (copying application source). We'll drill this hard in Milestone 5 & 6. For now the key fact is: **layers are content-addressed and cached, and one small change invalidates every layer after it**.

1.2.4.0.5 Image Digests (SHA256) and Immutability Every image and every layer is identified by a **digest** — a SHA256 hash of its content. You see them everywhere:

```
Digest: sha256:2140dad235c1dd3f4dc3ca35c7c8ca7b91c8f7c0a...
```

Digests are **immutable**. If two people around the world pull `nginx@sha256:2140dad...`, they are guaranteed byte-for-byte identical bits — the hash function guarantees it. This is why production deployments should pin to digests (`nginx@sha256:...`) rather than tags (`nginx:1.27`), because tags can be republished silently but digests cannot.

1.2.4.0.6 Tags — Human Names for Digests A **tag** is a mutable label that points to a digest. `nginx:1.27` today points to digest A; next week the Nginx team publishes 1.27.2 and republishes the tag pointing to digest B. Your image is fine on disk, but a fresh pull would grab B. This is why `nginx:latest` is dangerous in production — it moves every time upstream releases.

The full "image reference" format is:

```
[REGISTRY[:PORT]/]REPOSITORY[:TAG][@DIGEST]
```

Examples:

- `nginx` → `docker.io/library/nginx:latest`
- `nginx:1.27-alpine` → same registry, specific tag
- `ghcr.io/myorg/api:v2.3.1` → GitHub Container Registry
- `myregistry.corp:5000/team/tool:2024-11-05` → private registry on port 5000
- `nginx@sha256:2140dad235c1...` → tag omitted, digest specified

If you don't specify a registry, Docker defaults to `docker.io` (Docker Hub). If you don't specify a tag, Docker defaults to `:latest`. If you specify both a tag and a digest, the digest wins.

1.2.4.0.7 The `latest` Tag Is Just a Convention There is no "latest" magic in Docker — `latest` is simply the default tag assigned when no tag is given at push time. Some projects genuinely publish `latest` pointing to their most recent release; others do not. Never assume `latest` means "current" — always check what tag a project actually publishes. In production, always pin to a specific version.

1.2.4.0.8 Semantic Versioning in Image Tags Well-maintained projects follow **semantic versioning (semver)** in their tags:

- `nginx:1.27.2` — exact patch version
- `nginx:1.27` — latest 1.27.x
- `nginx:1` — latest 1.x.x
- `nginx:latest` — latest 1.x.x for stable releases; sometimes latest mainline
- `nginx:stable`, `nginx:mainline`, `nginx:perl` — flavor tags
- `nginx:1.27-alpine`, `nginx:1.27-slim` — same version, thinner base image

Choosing `nginx:1.27` gives you patch updates automatically on re-pull, which is usually what you want in staging; `nginx:1.27.2` pins you to an exact build for production reproducibility.

1.2.4.0.9 Docker Hub — The Default Registry **Docker Hub** (hub.docker.com) is the default public registry. It hosts millions of images in three main "spaces":

- **Official Images** — reviewed and maintained by Docker in cooperation with upstream projects. Namespace is `library/` (usually omitted). Examples: `nginx`, `python`, `postgres`, `alpine`, `debian`. Trustworthy.

- **Verified Publishers** — commercial vendors verified by Docker Inc. Examples: `bitnami/`, `cimg/` (CircleCI). Blue "Verified Publisher" badge.
- **Community Images** — anyone with a Docker Hub account. Namespace is `username/repo`. Quality ranges from excellent to abandoned. Always audit.

Docker Hub imposes **pull rate limits** on anonymous users (~100 pulls per 6 hours per IP), free logged-in users (~200 pulls per 6 hours), and higher limits for paid accounts. In corporate environments where many developers share a NAT'd egress IP, anonymous limits are frequently hit — that's a common cause of mysterious `too many requests` errors.

1.2.4.0.10 Identifying Trustworthy Images

When picking an image, look for:

1. **Official** badge (from Docker Hub), or a Verified Publisher badge.
2. **Last updated** date — abandoned images are a security risk.
3. **Pull count** — orders of magnitude matter. 10M+ pulls suggests significant use.
4. **Star count** — reasonable proxy for community trust.
5. **Documented Dockerfile** — a link to the source of the Dockerfile so you can inspect what actually goes into the image.
6. **CVE scan results** — Docker Scout, Trivy, or Snyk reports (Milestone 12 covers scanning).

For anything running in production, prefer official images or images from a Verified Publisher. For untrusted community images, always read the Dockerfile before pulling.

1.2.4.0.11 Docker Content Trust (DCT)

Docker Content Trust is an opt-in feature that uses cryptographic signatures to verify image integrity and publisher identity. Enable it by setting the environment variable:

```
$env:DOCKER_CONTENT_TRUST = "1"
```

With DCT on, `docker pull` and `docker push` reject unsigned images. It's built on **The Update Framework (TUF)** and used mostly by regulated industries and hardened corporate environments. In practice most teams instead rely on **image digest pinning** and **Sigstore/cosign** signatures (newer standard). DCT is worth knowing but not universally deployed.

Commands

- `docker pull` — download an image (covered in Milestone 3; revisited here for tags/digests).
- `docker images` / `docker image ls` — list images (covered in Milestone 3; revisited here with filters).
- `docker image inspect` — print full JSON metadata for an image.
- `docker history` — show the layer history of an image.
- `docker tag` — create a new tag pointing to an existing image.
- `docker search` — search Docker Hub for images from the CLI.
- `docker push` — upload a tagged image to a registry.
- `docker login` / `docker logout` — authenticate to a registry.
- `docker image prune` — remove dangling/unused images.
- `docker image save` / `docker image load` — export/import images as tarballs.

Command Options/Flags

1.2.4.0.12 `docker image inspect` flags

Flag	Short Form	Description	Example
`--format`	`-f`	Go template for output	`docker image inspect -f '{{.Config.Cmd}}' nginx`

1.2.4.0.13 `docker history` flags

Flag	Short Form	Description	Example
`--format`	—	Go template	`docker history --format '{{.CreatedBy}}' nginx`
`--human`	`-H`	Human-readable sizes (default true)	`docker history -H=false nginx`
`--no-trunc`	—	Show full commands (not truncated)	`docker history --no-trunc nginx`
`--quiet`	`-q`	Only show layer IDs	`docker history -q nginx`

1.2.4.0.14 `docker tag` flags

`docker tag` has no flags — it takes exactly two arguments:

```
docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
```

1.2.4.0.15 `docker search` flags

Flag	Short Form	Description	Example
`--filter`	`-f`	Filter results (is-official, stars)	`docker search -f is-official=true nginx`
`--format`	—	Go template	`docker search --format '{{.Name}}' nginx`
`--limit`	—	Max results (default 25)	`docker search --limit 5 nginx`
`--no-trunc`	—	Don’t truncate description	`docker search --no-trunc nginx`

1.2.4.0.16 `docker push` flags

Flag	Short Form	Description	Example
`--all-tags`	`-a`	Push all tagged variants	`docker push -a myuser/api`
`--disable-content-trust`	—	Skip content trust check	`docker push --disable-content-trust ...`
`--quiet`	`-q`	Suppress verbose output	`docker push -q myuser/api:1.0`

1.2.4.0.17 `docker login` flags

Flag	Short Form	Description	Example
`--username`	`-u`	Username	`docker login -u myuser`
`--password`	`-p`	Password (insecure — prefer stdin)	`docker login -p PASS`
`--password-stdin`	—	Read password from stdin	`echo $PAT | docker login -u me --password-stdin`

1.2.4.0.18 `docker image prune` flags

Flag	Short Form	Description	Example
`--all`	`-a`	Remove all unused (not just dangling)	`docker image prune -a`
`--force`	`-f`	No confirmation prompt	`docker image prune -f`
`--filter`	—	Filter which to prune	`docker image prune -a --filter "until=72h"`

1.2.4.0.19 `docker image save` / `docker image load` flags

Flag	Short Form	Description	Example
`--output`	`-o`	Save to a file (else stdout)	`docker save -o nginx.tar nginx:alpine`
`--input`	`-i`	Load from a file (else stdin)	`docker load -i nginx.tar`
`--quiet`	`-q`	Suppress output	`docker load -q -i nginx.tar`

1.2.4.0.20 `docker images` filter values (revisited)

Filter	Meaning	Example
`dangling=true`	Untagged (<none>:<none>) images	`docker images -f dangling=true`
`label=key` or `label=key=value`	Images with specific label	`docker images -f label=env=prod`
`before=IMAGE`	Older than given image	`docker images -f before=nginx:alpine`
`since=IMAGE`	Newer than given image	`docker images -f since=alpine:3.20`
`reference=PATTERN`	Repository/tag pattern (globs)	`docker images -f "reference=node:*-alpine"`

Examples

1.2.4.0.21 Example 1 — Inspect an image

```
docker pull nginx:alpine
docker image inspect nginx:alpine
```

```
[
    {
        "Id": "sha256:d1a364dc548d5357f0da3268c888e1971bbdb957ee3f028fe7194f1d61c6fdee",
        "RepoTags": ["nginx:alpine"],
        "RepoDigests": ["nginx@sha256:2140dad235c1..."],
        "Created": "2024-10-14T21:20:42Z",
        "Config": {
            "Env": ["PATH=/usr/local/sbin:...", "NGINX_VERSION=1.27.2", ...],
            "Cmd": ["nginx", "-g", "daemon off;"],
            "ExposedPorts": {"80/tcp": {}},
            "WorkingDir": "",
            "Entrypoint": ["/docker-entrypoint.sh"],
            "StopSignal": "SIGQUIT"
        },
        "Architecture": "amd64",
        "Os": "linux",
        "Size": 20504389,
        "RootFS": {
```

```
            "Type": "layers",
            "Layers": [
                "sha256:c6b39d5da...", "sha256:27f5a1b7e...", ...
            ]
        }
    }
]
```

Key fields:

- `Id` — the image's own SHA256.
- `RepoTags` — all human-readable tags pointing at this image on the local machine.
- `RepoDigests` — the pull-time digest as fetched from the registry.
- `Created` — image build timestamp.
- `Config.Env` — default environment variables inside the container.
- `Config.Cmd` — the default command that runs when a container is started.
- `Config.ExposedPorts` — declared listening ports (informational; does not auto-publish).
- `Config.Entrypoint` — the wrapper script (Nginx's entrypoint copies configs into place before exec'ing the CMD).
- `Config.StopSignal` — signal sent by `docker stop` (Nginx uses `SIGQUIT` for graceful shutdown).
- `Architecture` / `Os` — target platform.
- `Size` — total on-disk size in bytes.
- `RootFS.Layers` — list of layer digests in order (bottom → top).

1.2.4.0.22 Example 2 — Extract just one field with `--format`

```
docker image inspect --format '{{.Config.Cmd}}' nginx:alpine
docker image inspect --format '{{.Architecture}}/{{.Os}}' nginx:alpine
docker image inspect --format '{{.Size}}' nginx:alpine
```

```
[nginx -g daemon off;]
amd64/linux
20504389
```

1.2.4.0.23 Example 3 — Layer history

```
docker history nginx:alpine
```

```
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
d1a364dc548d   2 weeks ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfi
<missing>      2 weeks ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v
<missing>      2 weeks ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerf
<missing>      2 weeks ago   COPY 30-tune-worker-processes.sh                4.62kB    buildkit.dockerf
<missing>      2 weeks ago   COPY docker-entrypoint.sh                       1.62kB    buildkit.dockerfil
<missing>      2 weeks ago   RUN /bin/sh -c set -x  && addgroup -g 101 ...   12.4MB    buildkit.docker
<missing>      2 weeks ago   ENV DYNPKG_RELEASE=1                            0B        buildkit.dockerfile.v
...
<missing>      3 weeks ago   ADD alpine-minirootfs.tar.gz                    7.7MB     buildkit.dockerfil
```

Reading top-to-bottom: the newest layer (which produced this image ID) is at the top. Each row is one instruction from the Dockerfile. `<missing>` in the IMAGE column just means those intermediate layers were not preserved with their own image IDs (normal for pulled images). Total size ≈ sum of the SIZE column.

1.2.4.0.24 Example 4 — Tag an image with a new name

```
docker tag nginx:alpine myusername/nginx-custom:v1.0
docker images | Select-String nginx
```

```
myusername/nginx-custom   v1.0     d1a364dc548d   2 weeks ago   20.5MB
nginx                     alpine   d1a364dc548d   2 weeks ago   20.5MB
```

Notice both tags have the *same IMAGE ID* — `docker tag` did not duplicate anything on disk. A tag is just a pointer.

1.2.4.0.25 Example 5 — Search Docker Hub

```
docker search --limit 5 --filter is-official=true python
```

```
NAME     DESCRIPTION                              STARS     OFFICIAL
python   Python is an interpreted, ...            9345      [OK]
```

1.2.4.0.26 Example 6 — Push (requires login)

```
docker login
docker push myusername/nginx-custom:v1.0
```

```
Login Succeeded

The push refers to repository [docker.io/myusername/nginx-custom]
c6b39d5da001: Mounted from library/nginx
27f5a1b7e34c: Mounted from library/nginx
...
v1.0: digest: sha256:2140dad235c1... size: 1741
```

`Mounted from library/nginx` means the registry already had those layers from the official `nginx` repo and simply linked them — no re-upload.

1.2.4.0.27 Example 7 — Enable Docker Content Trust

```
$env:DOCKER_CONTENT_TRUST = "1"
docker pull alpine:3.20
```

```
Pull (1 of 1): alpine:3.20@sha256:....
3.20@sha256:...: Pulling from library/alpine
...
Tagging alpine@sha256:... as alpine:3.20
```

When DCT is on, Docker refuses to pull unsigned images. Disable with:

```
$env:DOCKER_CONTENT_TRUST = ""
```

Command Exercises

1.2.4.0.28 Exercise 4.1 — `docker image inspect` basic Inspect the `alpine:3.20` image and identify its size in bytes and its default `CMD`.

▶ Solution

```
docker pull alpine:3.20
docker image inspect alpine:3.20
```

```
[
    {
        "Id": "sha256:71fc0a10c1a1...",
        "Size": 7700000,
        "Config": {
            "Cmd": ["/bin/sh"],
            ...
        },
        ...
    }
]
```

Answers: Size ≈ 7.7 MB (the smallest general-purpose Linux distro), CMD is `/bin/sh` — which is why `docker run -it alpine` drops you at a shell without specifying a command.

1.2.4.0.29 Exercise 4.2 — `docker image inspect --format` Print exactly one line containing the exposed ports of `nginx:alpine`.

▶ Solution

```
docker image inspect --format '{{.Config.ExposedPorts}}' nginx:alpine
```

```
map[80/tcp:{}]
```

Go template renders the map as Go syntax. To get a cleaner display:

```
docker image inspect --format '{{range $p, $_ := .Config.ExposedPorts}}{{$p}} {{end}}' nginx
```

```
80/tcp
```

1.2.4.0.30 Exercise 4.3 — `docker image inspect` combined

For every image locally cached, print `name:tag -> architecture -> size(MB)`.

Solution

```
foreach ($id in (docker images -q)) {
  docker image inspect --format '{{index .RepoTags 0}} -> {{.Architecture}} -> {{.Size}}' $id
}
```

```
nginx:alpine -> amd64 -> 20504389
alpine:3.20 -> amd64 -> 7700000
python:3.12-slim -> amd64 -> 129000000
...
```

Convert bytes to MB in the shell if you want:

```
foreach ($id in (docker images -q)) {
  $t = docker image inspect --format '{{index .RepoTags 0}}' $id
  $s = [int](docker image inspect --format '{{.Size}}' $id)
  "$t -> $([math]::Round($s/1MB,1)) MB"
}
```

```
nginx:alpine -> 19.6 MB
alpine:3.20 -> 7.3 MB
python:3.12-slim -> 123 MB
```

1.2.4.0.31 Exercise 4.4 — `docker history` basic

Show the layer history of `alpine:3.20`.

Solution

```
docker history alpine:3.20
```

```
IMAGE          CREATED       CREATED BY                      SIZE     COMMENT
71fc0a10c1a1   2 weeks ago   CMD ["/bin/sh"]                 0B       buildkit.dockerfile.v0
<missing>      2 weeks ago   ADD alpine-minirootfs.tar.gz    7.7MB    buildkit.dockerfile.v0
```

Alpine is only two layers — that's why it's tiny. Compare to Ubuntu or Debian which have many more.

1.2.4.0.32 Exercise 4.5 — `docker history --no-trunc`

Reveal the FULL Dockerfile commands (not truncated) for `nginx:alpine`.

Solution

```
docker history --no-trunc nginx:alpine
```

```
IMAGE     CREATED       CREATED BY                                                      SIZE
sha256... 2 weeks ago   CMD ["nginx" "-g" "daemon off;"]                                  0B
<missing> 2 weeks ago   RUN /bin/sh -c set -x     && addgroup -g 101 -S nginx     &&
                        adduser -S -D -H -u 101 -h /var/cache/nginx -s /sbin/nologin
                        -G nginx -g nginx nginx     && apkArch="$(cat /etc/apk/arch)"
                      ...                                                             12.4MB
...
```

You now see the exact shell command that produced each layer. `--no-trunc` is essential when reverse-engineering an image.

1.2.4.0.33 Exercise 4.6 — `docker history` size analysis

For `nginx:alpine`, identify the SINGLE largest layer and its size.

Solution

```
docker history nginx:alpine --format '{{.Size}}\t{{.CreatedBy}}' | Sort-Object { [regex]::Ma
```

```
12.4MB   RUN /bin/sh -c set -x && addgroup -g 101 ...    (the nginx install)
7.7MB    ADD alpine-minirootfs.tar.gz ...                (the alpine base)
4.62kB   COPY 30-tune-worker-processes.sh ...
```

Answer: the largest single layer is the RUN step that installs nginx itself (~12.4 MB). This is typical — the RUN that installs packages is almost always the biggest layer in a Dockerfile.

1.2.4.0.34 Exercise 4.7 — `docker search` basic Search Docker Hub for images related to `postgres`, limited to 5 results.

 Solution

```
docker search --limit 5 postgres
```

```
NAME                 DESCRIPTION                              STARS   OFFICIAL
postgres             The PostgreSQL object-relational...      13209   [OK]
bitnami/postgresql   Bitnami PostgreSQL Docker Image          278
postgrest/postgrest  REST API for any Postgres database       158
circleci/postgres    CircleCI images ...                       32
kartoza/postgis      PostGIS spatial extension                189
```

1.2.4.0.35 Exercise 4.8 — `docker search` filtered Search for `python` images, official only.

 Solution

```
docker search --filter is-official=true python
```

```
NAME     DESCRIPTION                              STARS   OFFICIAL
python   Python is an interpreted, ...           9345    [OK]
```

Only the official `library/python` image comes back. The `is-official` filter is the fastest way to find trustworthy base images from the CLI.

1.2.4.0.36 Exercise 4.9 — `docker search` — troubleshooting a garbage result Search for `nodejs` and note how many results are from user accounts (not official). What is the correct official image name?

 Solution

```
docker search --limit 10 nodejs
```

```
NAME                    DESCRIPTION                              STARS   OFFICIAL
adhawk/nodejs           ...                                      5
seovaia/nodejs          ...                                      3
...
```

None marked OFFICIAL — because the official image is named `node`, not `nodejs`. Correct search:

```
docker search --filter is-official=true node
```

```
NAME   DESCRIPTION                              STARS   OFFICIAL
node   Node.js is a JavaScript-based platform  13897   [OK]
```

Lesson: image names don't always match the "friendly" project name. Verify official image names at hub.docker.com before adopting them.

1.2.4.0.37 Exercise 4.10 — `docker tag` basic Tag the local `alpine:3.20` image as `mylabs/alpine:2025-11-05`.

 Solution

```
docker tag alpine:3.20 mylabs/alpine:2025-11-05
docker images | Select-String -Pattern "alpine|mylabs"
```

```
mylabs/alpine     2025-11-05   71fc0a10c1a1   2 weeks ago   7.7MB
alpine            3.20         71fc0a10c1a1   2 weeks ago   7.7MB
```

Same IMAGE ID — just a new pointer.

1.2.4.0.38 Exercise 4.11 — `docker tag` for registry push Tag `alpine:3.20` for a private registry at `myregistry.corp:5000` under `infra/alpine` with tag `latest`.

 Solution

```
docker tag alpine:3.20 myregistry.corp:5000/infra/alpine:latest
docker images myregistry.corp:5000/infra/alpine
```

```
REPOSITORY                              TAG       IMAGE ID       CREATED         SIZE
myregistry.corp:5000/infra/alpine       latest    71fc0a10c1a1   2 weeks ago     7.7MB
```

Including the registry host (with port) in the tag is how Docker knows where to push. Without the host, it defaults to Docker Hub.

1.2.4.0.39 Exercise 4.12 — `docker tag` — undo by untagging Remove ONLY the tag `mylabs/alpine:2025-11-05` without deleting the underlying image.

 Solution

```
docker rmi mylabs/alpine:2025-11-05
docker images | Select-String -Pattern "alpine|mylabs"
```

```
Untagged: mylabs/alpine:2025-11-05

alpine   3.20   71fc0a10c1a1   2 weeks ago   7.7MB
```

Because `alpine:3.20` still points to the same image ID, Docker only removes the extra tag, not the layers. This is called “untagging” — distinct from actually deleting layers, which only happens when the last tag/reference is removed.

1.2.4.0.40 Exercise 4.13 — `docker images` filter by reference List all locally cached Node.js images with a specific pattern.

 Solution

```
docker pull node:20-alpine
docker pull node:20-slim
docker images -f "reference=node:*-alpine"
```

```
REPOSITORY   TAG          IMAGE ID       CREATED       SIZE
node         20-alpine    a1b2c3d4e5f6   2 weeks ago   135MB
```

The `reference` filter accepts glob patterns matching `repo:tag`.

1.2.4.0.41 Exercise 4.14 — `docker images` filter dangling Create a dangling image and then list dangling images.

 Solution

```
# Build once
mkdir tmpbuild; cd tmpbuild
"FROM alpine:3.20`nRUN echo v1 > /tag.txt" | Out-File Dockerfile -Encoding ascii
docker build -t demo:v1 .

# Change and rebuild — the previous demo:v1 becomes dangling
"FROM alpine:3.20`nRUN echo v2 > /tag.txt" | Out-File Dockerfile -Encoding ascii
docker build -t demo:v1 .

docker images -f dangling=true
```

```
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
<none>       <none>    abc123def456   30 seconds ago   7.7MB
```

The dangling image is the “orphaned” <none>:<none> version that no longer has a tag pointing at it (because the second build stole the `demo:v1` tag). Clean up:

```
docker image prune -f
```

1.2.4.0.42 Exercise 4.15 — `docker images` filter since/before List all images pulled AFTER `alpine:3.20`.

 Solution

```
docker images -f since=alpine:3.20
```

```
REPOSITORY   TAG          IMAGE ID       CREATED           SIZE
node         20-alpine    a1b2c3d4e5f6   30 seconds ago    135MB
demo         v1           efabcd012345   3 minutes ago     7.7MB
```

Useful for identifying “what have I pulled today” or clearing recent builds.

Hands-On Assignment

Task: Pull three variants of Node.js 20, inspect them, and produce a written comparison.

Steps:

1. Pull all three variants:

   ```
   docker pull node:20
   docker pull node:20-slim
   docker pull node:20-alpine
   ```

2. List them with size:

   ```
   docker images node --format 'table {{.Tag}}\t{{.Size}}'
   ```

   ```
   TAG          SIZE
   20           1.11GB
   20-slim      241MB
   20-alpine    135MB
   ```

3. Inspect each and record the base OS. Hint: check the layer history for the base image ADD/tar.gz layer, or read the label `org.opencontainers.image.base.name` if present:

   ```
   docker image inspect --format '{{index .Config.Labels "org.opencontainers.image.base.nam
   docker image inspect --format '{{index .Config.Labels "org.opencontainers.image.base.nam
   docker image inspect --format '{{index .Config.Labels "org.opencontainers.image.base.nam
   ```

4. Count layers each:

   ```
   docker history node:20 -q | Measure-Object | ForEach-Object Count
   docker history node:20-slim -q | Measure-Object | ForEach-Object Count
   docker history node:20-alpine -q | Measure-Object | ForEach-Object Count
   ```

5. Create a comparison table in a `nodejs-comparison.md` file:

Tag	Base	Size	Layers
node:20	debian:bookworm	1.11 GB	8
node:20-slim	debian:bookworm-slim	241 MB	5
node:20-alpine	alpine:3.20	135 MB	4

Acceptance criteria:

- ☐ All three images pulled and visible in `docker images`.
- ☐ Comparison table saved to `nodejs-comparison.md`.
- ☐ Base OS identified for each variant.
- ☐ Layer counts recorded.
- ☐ You can state, in one sentence, which variant you’d pick for production and why.

Mini-Project

1.2.4.0.43 Project Title Docker Image Investigator Report

1.2.4.0.44 Objective Choose a category of application (web servers), pull five popular images in that category, forensically analyze each using `inspect`, `history`, and `search` metadata, then produce a written recommendation for which to use in a hypothetical production deployment. This is the exact workflow a platform engineer performs when choosing base images for their team.

1.2.4.0.45 Requirements

1. **Pick the category:** web servers.
2. **Pull five official/community images:** `nginx:alpine`, `httpd:alpine`, `caddy:2-alpine`, `traefik:v3.1`, `lighttpd`.
3. **For each image, record:** size, layer count, base OS (from labels or last ADD layer), exposed ports, entrypoint, cmd, and creation date.
4. **Build a comparison table** with columns: Image, Size, Layers, Base, Exposed Ports, Entrypoint, CMD, Created.
5. **Identify the largest single layer** in each image using `docker history`.
6. **Tag one image** with a custom tag `mycompany/webserver:2025-11-eval` and verify with `docker images`.
7. **(Optional)** Push the tagged image to your personal Docker Hub account (skip if you don't have one — no penalty).
8. **Write a recommendation:** given a hypothetical requirement — "we need a reverse proxy in front of 20 internal microservices, minimizing image size and CVE surface, with built-in HTTPS/Let's Encrypt" — which image would you pick and why?

1.2.4.0.46 Step-by-Step Guidance

1. Create folder `docker-mastery\milestone-4\`, then file `image-investigator.md`.
2. In the file, write a `## Setup` section listing the five `docker pull` commands and their outputs.
3. Write a `## Per-Image Analysis` section. For each image, subsection with:
   - `### <image>`
   - Output of `docker image inspect --format '<template>'` for size, cmd, entrypoint, exposed ports, created, architecture.
   - Output of `docker history <image>` — full table.
   - A one-sentence summary of the largest layer.
4. Write a `## Comparison Table` section with all five images side-by-side.
5. Write a `## Tagging Demo` section — the `docker tag` command and its verification via `docker images`.
6. Write a `## Recommendation` section (200–400 words) with your pick, justification, and trade-offs.
7. (Optional) Write a `## Push Log` section if you pushed to Docker Hub.

1.2.4.0.47 Complete Mini-Project Solution Complete Mini-Project Solution

```
# Docker Image Investigator Report — Web Servers

## Setup — Pull five images

```powershell
docker pull nginx:alpine
docker pull httpd:alpine
docker pull caddy:2-alpine
docker pull traefik:v3.1
docker pull lighttpd
```

Per-Image Analysis

nginx:alpine

```powershell
docker image inspect --format 'Size:{{.Size}} | Cmd:{{.Config.Cmd}} | Entrypoint:{{.Config.E
docker history nginx:alpine
```

- Size: ~20.5 MB
- Base: alpine:3.20
- Exposed: 80/tcp
- Entrypoint: [/docker-entrypoint.sh]
- Cmd: [nginx -g daemon off;]
- Layers: 9
- Largest layer: 12.4 MB (RUN installing nginx package)
```

httpd:alpine

- Size: ~62 MB
- Base: alpine:3.20
- Exposed: 80/tcp
- Entrypoint: [httpd-foreground]
- Cmd: []
- Layers: 8
- Largest layer: ~44 MB (RUN compile-and-install httpd)

caddy:2-alpine

- Size: ~52 MB
- Base: alpine:3.20
- Exposed: 80/tcp, 443/tcp, 443/udp, 2019/tcp
- Entrypoint: [caddy]
- Cmd: [run --config /etc/caddy/Caddyfile --adapter caddyfile]
- Layers: 6
- Largest layer: ~44 MB (RUN adding caddy binary)

traefik:v3.1

- Size: ~172 MB
- Base: alpine (no explicit label)
- Exposed: 80/tcp
- Entrypoint: [/entrypoint.sh]
- Cmd: [traefik]
- Layers: 7
- Largest layer: ~165 MB (COPY traefik binary + plugins)

lighttpd

- Size: ~14 MB
- Base: alpine:3.20
- Exposed: 80/tcp
- Entrypoint: []
- Cmd: [lighttpd -D -f /etc/lighttpd/lighttpd.conf]
- Layers: 5
- Largest layer: ~6 MB (RUN install lighttpd)

Comparison Table

Image	Size	Layers	Base	Ports	Entrypoint
nginx:alpine	20.5 MB	9	alpine	80	/docker-entrypoint.s
httpd:alpine	62 MB	8	alpine	80	httpd-foreground
caddy:2-alpine	52 MB	6	alpine	80, 443, 443/udp	caddy
traefik:v3.1	172 MB	7	alpine	80	/entrypoint.sh
lighttpd	14 MB	5	alpine	80	(none)

Tagging Demo

```powershell
docker tag caddy:2-alpine mycompany/webserver:2025-11-eval
docker images | Select-String mycompany
```

Output:
```
mycompany/webserver   2025-11-eval   0d1e2f3a4b5c   2 weeks ago   52MB
```

```
## Recommendation

For the stated requirement — a reverse proxy in front of 20 internal
microservices, minimizing image size and CVE surface, with built-in
HTTPS/Let's Encrypt — my pick is **Caddy (`caddy:2-alpine`)**.

Rationale:

1. **Built-in HTTPS via ACME.** Caddy handles Let's Encrypt certificate
   provisioning and renewal automatically with zero configuration. Nginx
   and Traefik can do it but require plugin/config work; Apache and
   Lighttpd require external tooling like certbot.
2. **Small footprint.** At ~52 MB Caddy is 3x smaller than Traefik and
   about 2.5x larger than nginx:alpine — a reasonable trade-off given
   the HTTPS features it includes out of the box.
3. **Modern config (Caddyfile).** Much easier to read and maintain for
   20 microservices than Nginx conf blocks.
4. **Single static Go binary.** Fewer runtime dependencies → smaller
   attack surface than Apache's module ecosystem.

Trade-offs I would flag to my team:

- Caddy has a smaller operator community than Nginx or Traefik. If the
  team already has deep Nginx expertise, Nginx + certbot may be a lower
  learning-curve choice.
- If we later need active service discovery (Docker Swarm / K8s labels),
  Traefik is superior — I would revisit then.
- If image size is *strictly* the top priority, `nginx:alpine` (20 MB)
  or `lighttpd` (14 MB) win, but you lose auto-HTTPS.

Second choice: `nginx:alpine` for its long-term stability and huge
community. Third choice: `traefik:v3.1` if service discovery becomes
a requirement.
```

1.2.4.0.48 ✅ Verification Checklist

- ☐ `image-investigator.md` exists at `docker-mastery\milestone-4\`.
- ☐ All five images (`nginx:alpine`, `httpd:alpine`, `caddy:2-alpine`, `traefik:v3.1`, `lighttpd`) are pulled — verified via `docker images`.
- ☐ For each image, the report records: size, base OS, exposed ports, entrypoint, cmd, created date, layer count.
- ☐ `docker image inspect --format` was used at least once per image.
- ☐ `docker history` was inspected for each image and the largest layer is called out.
- ☐ A comparison table with all five images appears in the report.
- ☐ `docker tag` was used to create `mycompany/webserver:2025-11-eval` and verified.
- ☐ A written recommendation paragraph (200+ words) explains a preferred choice with rationale and trade-offs.
- ☐ Report includes at least three distinct considerations (size, security, features, ease of use) in the recommendation.

1.2.4.0.49 🎯 Bonus Challenges

1. **Multi-arch check.** Use `docker manifest inspect` (experimental) on each of the five to list which platforms each image publishes. Add a "Multi-arch support" column to your comparison table.
2. **CVE preview.** If you have Docker Scout enabled, run `docker scout cves nginx:alpine` and record the number of high/critical CVEs for each image. Add a "CVE count" column and let that influence your recommendation.
3. **Push to Docker Hub.** Sign up for a free Docker Hub account, `docker login`, and push the `mycompany/webserver:2025-11-eval` tag to your namespace. Add the "Push Log" section to your report showing the layer-mount optimization at work.

Scenario (Real-World Use Case)

You've joined a platform team responsible for approving base images used by 200 developers across

40 microservices. Marketing announces a new mobile push notification service, and its lead engineer files a ticket: "We want to use `randomuser/node-alpine-fast:latest` as our base — it's smaller and starts faster."

Before approving, you run through Milestone 4 muscle memory: `docker pull randomuser/node-alpine-fast:latest`, then `docker image inspect` reveals the image was built two years ago. `docker history --no-trunc` shows a suspicious `curl | bash` step downloading a script from a shortener URL. `docker search randomuser/node-alpine-fast` shows 3 stars and no verified publisher badge. You reject the image, point the engineer at the official `node:20-alpine`, and explain the risk in three bullet points that reference the exact `docker history` output.

Your team writes a policy: "All base images must be Official or Verified Publisher, updated within the last 90 days, with no external downloads in RUN steps." You put a Slack bot on it that automates the check. Every skill from Milestone 4 — `inspect`, `history`, `search`, understanding tags vs digests, official vs community — was in play in that 20-minute review.

Checkpoint Quiz

Question 1. What is the difference between a tag and a digest?

Click to reveal answer

A **tag** is a mutable, human-readable label (like `nginx:1.27`) that a publisher can repoint to a different image at any time. A **digest** is an immutable SHA256 hash (like `nginx@sha256:2140dad235c1...`) computed from the image's exact contents. Two pulls of the same digest, anywhere on Earth, are guaranteed byte-identical. Production deployments should pin to digests for reproducibility; tags are fine for development where you want automatic updates.

Question 2. If two different images both start with `FROM python:3.12-slim`, how many copies of the `python:3.12-slim` layers does Docker store on your disk?

Click to reveal answer

One copy. Docker stores each layer once (keyed by content hash) and multiple images reference the same layer blob. That is the entire point of the layered filesystem — massive disk savings across related images. This is also why pulling many images from the same base is fast: only the differing top layers need to download.

Question 3. Predict: what does `docker tag foo:1.0 foo:2.0` do to disk usage?

Click to reveal answer

Nothing. `docker tag` only creates a new name pointing to an existing image ID — no layers are copied, no bytes hit the disk. This is why tagging is instant and free. Contrast this with `docker build` or `docker commit`, which produce new layer content and do consume disk.

Question 4. Your CI pipeline pulls `nginx:1.27` on every build. Six weeks later, `nginx:1.27` is silently republished by the Nginx team with a security patch (`1.27.2`). What happens on your next CI build?

Click to reveal answer

Docker sees the tag `nginx:1.27` now points to a new digest. On pull, Docker downloads the new layers and updates your local cache. Your build starts using the patched image — invisibly and automatically. This is often what you want (auto-patching), but it means the same tag can yield different builds over time. To lock in exact reproducibility, pin to `nginx@sha256:...` instead of `nginx:1.27`.

Question 5. You run `docker history someimage` and see one row `<missing>` with a 2 GB size. What does `<missing>` mean, and why is the size that big?

Click to reveal answer

`<missing>` in the `IMAGE` column means that intermediate layer was **not stored with its own tagged image ID** — normal for images you pulled (rather than built locally). It does *not* mean the layer is broken. The 2 GB size means whatever Dockerfile instruction produced that layer wrote 2 GB of content — often a `RUN apt-get install ... -y` for something heavy like TeX Live or CUDA, or a `COPY` of a giant asset. Big layers are red flags for image bloat and a signal to investigate multi-stage builds (Milestone 6).

⬆ Back to Table of Contents

End of Part 1. Part 2 (Milestones 5–8) will continue with Writing Dockerfiles, Building & Optimizing Images, Container Data Management, and Docker Networking.